BCSE205L
Computer Architecture and Organization

Module 7 – High Performance Processors

Module:7	High Performance Processors	7 Hours
Classification of models - Flynn's taxonomy of parallel machine models (SISD, SIMD, MISD, MIMD) - Pipelining: Two stages, Multi stage pipelining, Basic performance issues in pipelining, Hazards, Methods to prevent and resolve hazards and their drawbacks - Approaches to deal branches - Superscalar architecture: Limitations of scalar pipelines, superscalar versus super pipeline architecture, superscalar techniques, performance evaluation of superscalar architecture - performance evaluation of parallel processors: Amdahl's law, speed-up and efficiency.		

Dr. P.Keerthika

Associate Professor

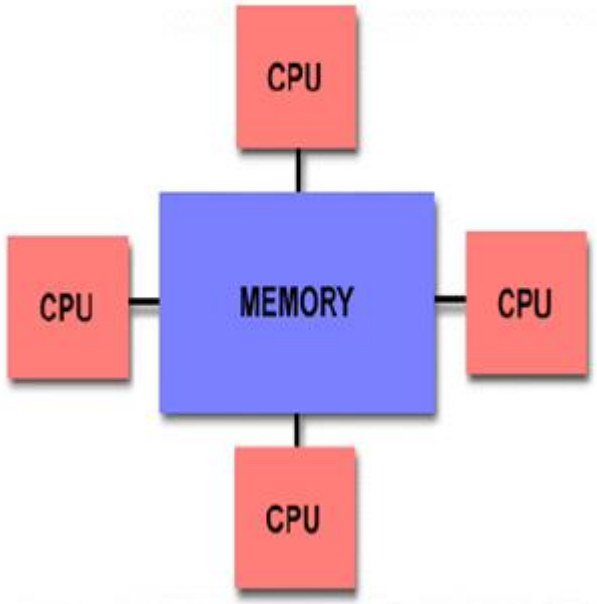
School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

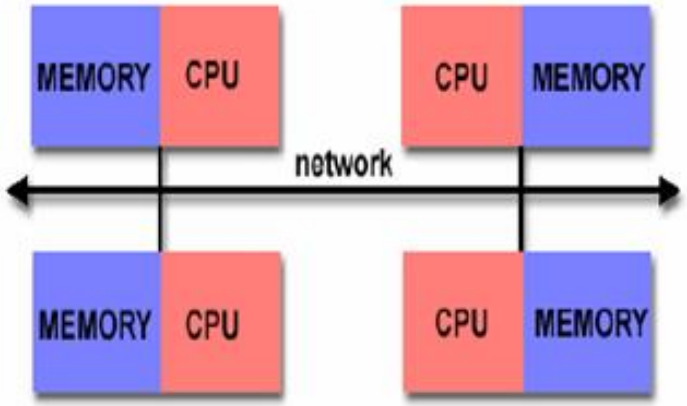
Parallel Processing

- Parallel computers are those that emphasize the parallel processing between the operations in some way.
- Various Varieties
  - Shared Memory
  - Distributed Memory
  - Hybrid Memory

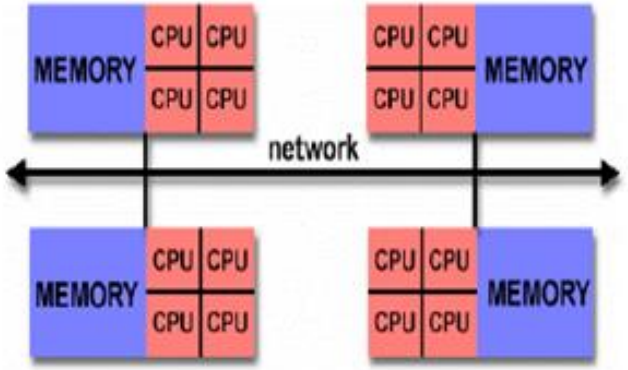
Shared Memory



Distributed Memory

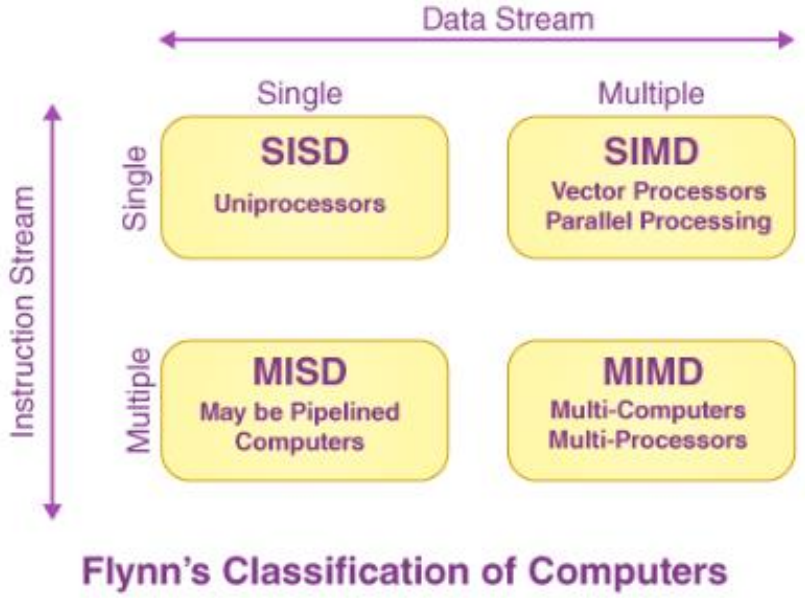


Hybrid Memory



Classification of Parallel Machine models - Flynn's Taxonomy

- This classification was first studied and proposed by Michael Flynn in 1972.
- Flynn did not consider the machine architecture for classification of parallel computers
- He introduced the concept of instruction and data streams for categorizing of computers.
- Instruction stream - a flow of instructions from main memory to the CPU
- Data stream - a flow of operands between processor and memory
- All the computers classified by Flynn are not parallel computers
- Let I_s and D_s are minimum number of streams flowing at any point in the execution



Flynn's Classification of Computers

Classification of Parallel Machine models - Flynn's Taxonomy

Instruction Cycle

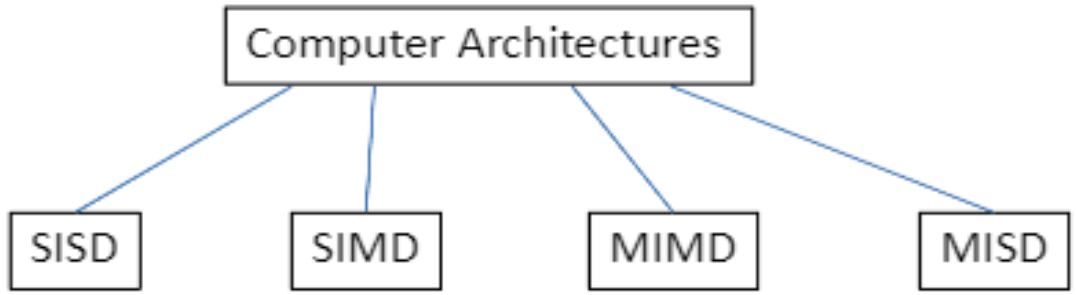
- Start
- Calculate the address of the instruction to be executed
- Fetch the instruction
- Decode the instruction
- Calculate the operand address
- Fetch the operands
- Execute the instructions
- Store the results
- If more instructions to be executed, go to step 2 else stop.

Flynn's Taxonomy of Parallel Machine models

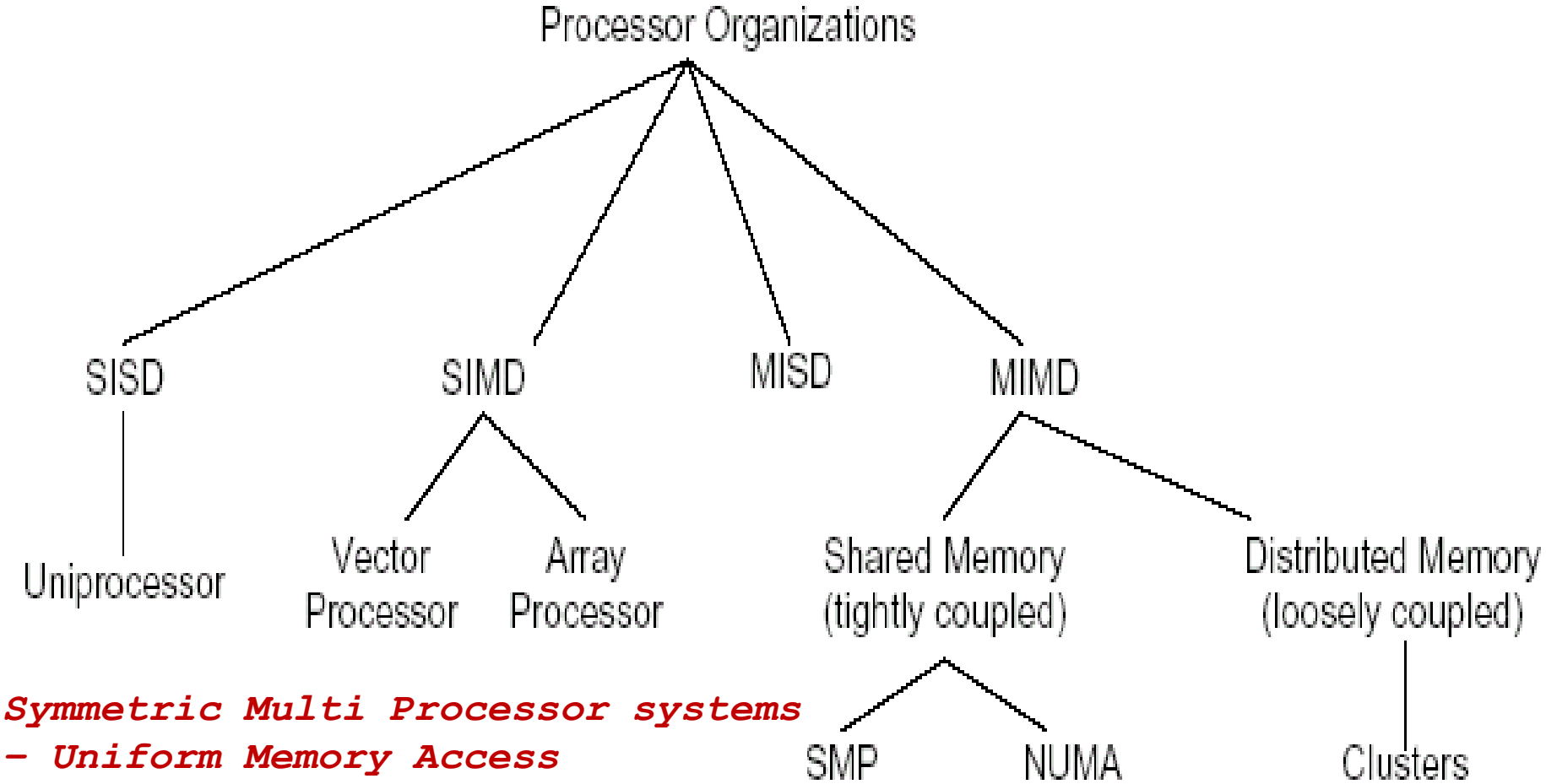
- Single Instruction Single Data (SISD)
  - Traditional sequential computing systems
- Single Instruction Multiple Data (SIMD)
- Multiple Instructions Multiple Data (MIMD)
- Multiple Instructions Single Data (MISD)

Classical Von-Neumann Architecture

Most Common and general Parallel Machine

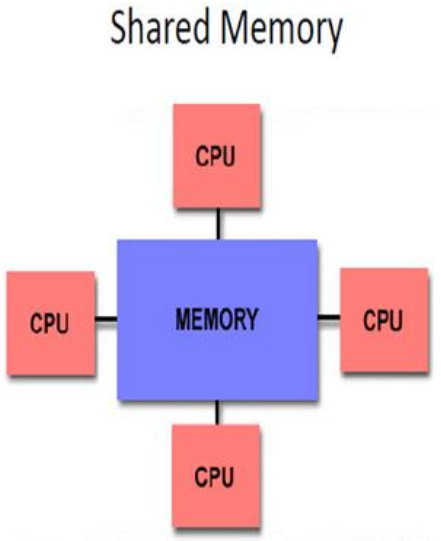


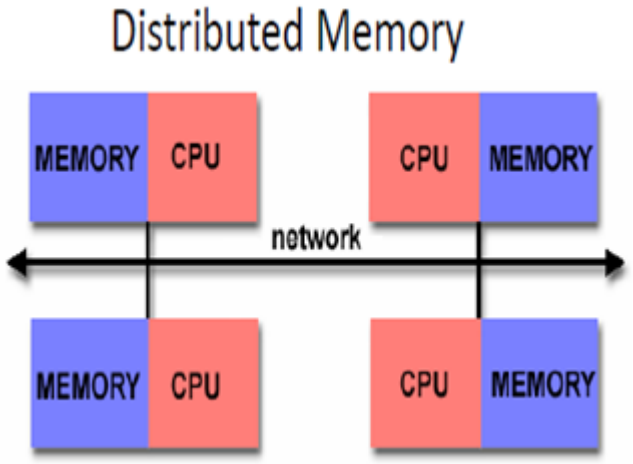
Flynn's Taxonomy of Parallel Machine models



Symmetric Multi Processor systems - Uniform Memory Access

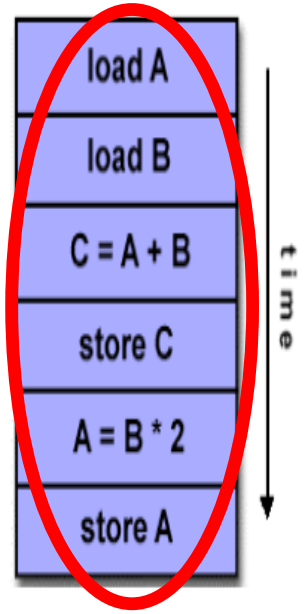
Non - Uniform Memory Access



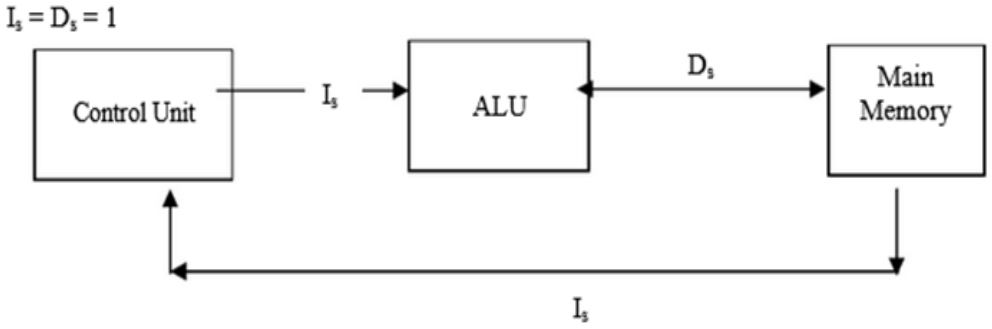


Flynn's Taxonomy - SISD

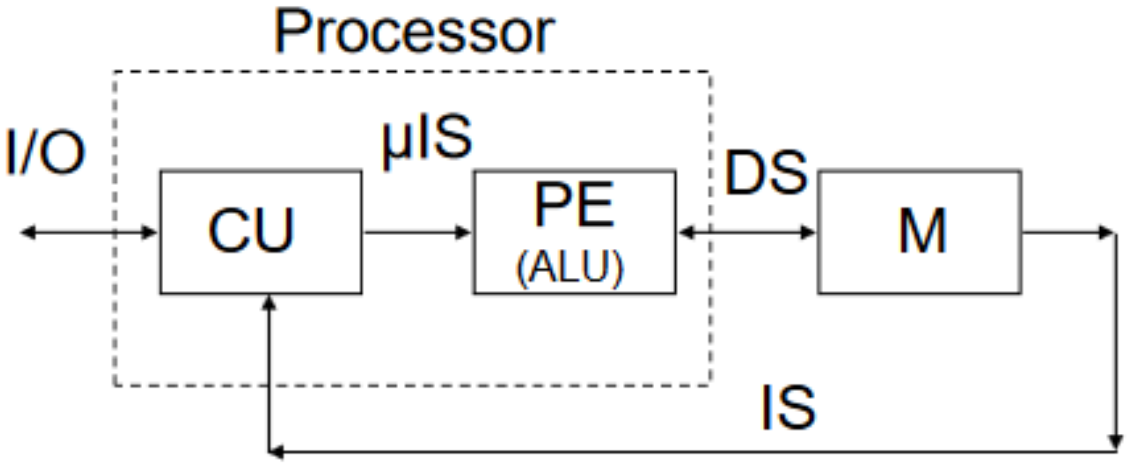
- A serial computer
- SISD machines are conventional serial computers that process only one stream of instructions and one stream of data.
- $I_S = D_S = 1$
- Examples
  - CDC 6600 which is unpipelined but has multiple functional units.
  - CDC 7600 which has a pipelined arithmetic unit.
  - Amdhal 470/6 which has pipelined instruction processing.
  - Cray-1 which supports vector processing.

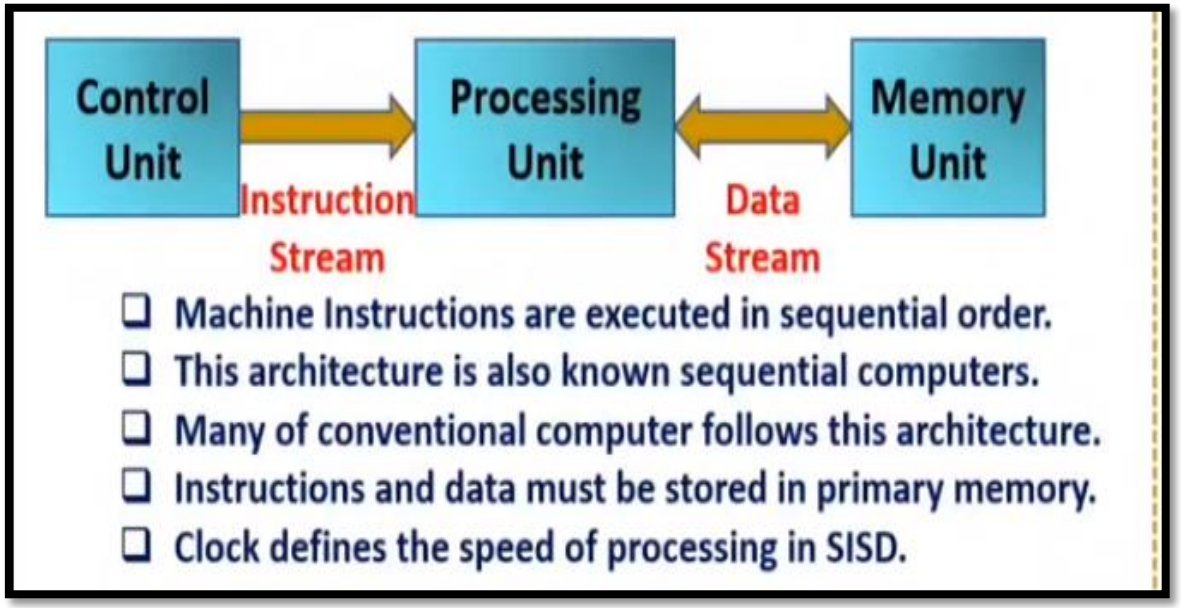


- An SISD computing system -**uniprocessor** machine - capable of **executing a single instruction, operating on a single data stream.**
- In SISD, machine **instructions are processed in a sequential manner** and computers adopting this model are popularly called **sequential computers.**
- Most conventional computers have SISD architecture.
- **All the instructions and data to be processed have to be stored in primary memory**.



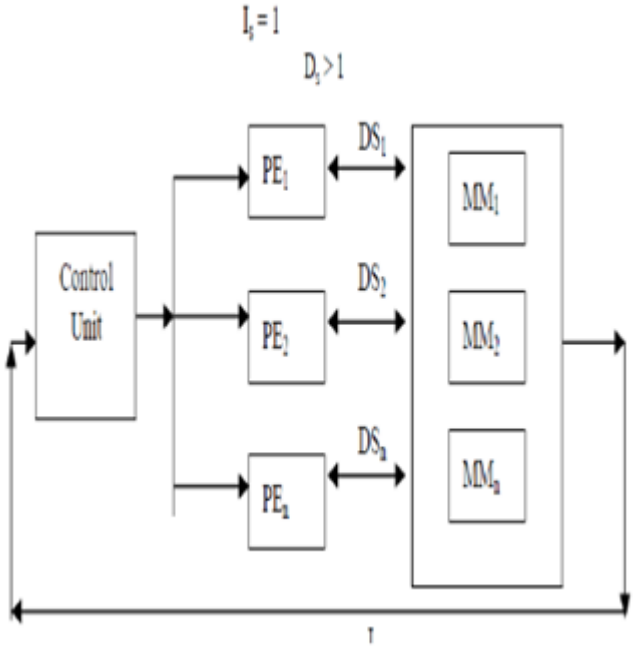
Flynn's Taxonomy - SISD



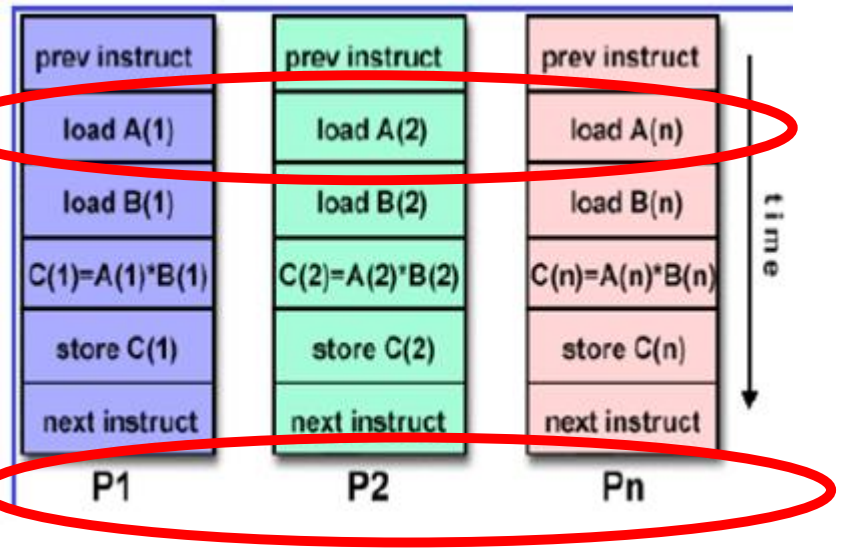


This is the uniprocessor architecture
CU= Control Unit
PE= Processing Element (same as ALU)
M= Memory
IS= Instruction Stream
DS= Data Stream
μIS= Micro-Instructions Stream

Flynn's Taxonomy - SIMD

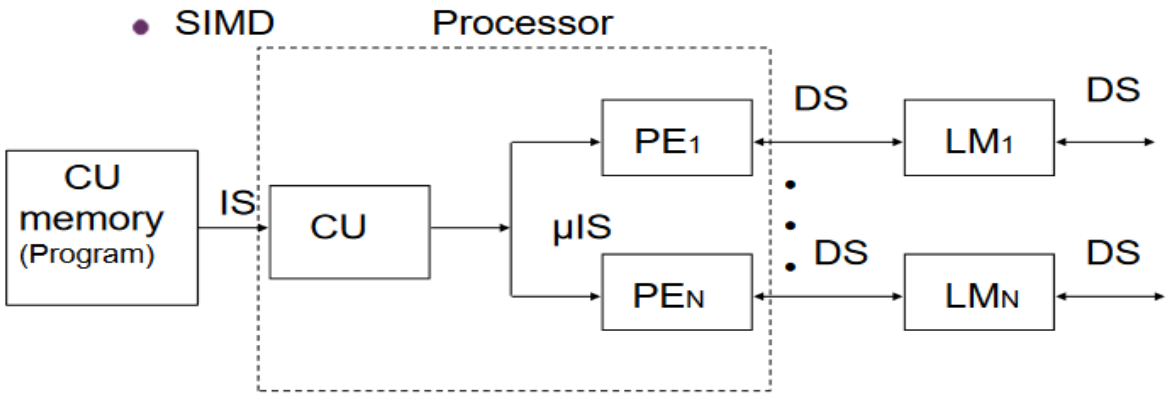


- Multiple processing elements work under the control of a single control unit.
- It has one instruction and multiple data stream
- Main memory can also be divided into modules for generating multiple data streams acting as a distributed memory
- Examples of SIMD organisation are ILLIAC-IV, PEPE, BSP, STARAN, MPP, DAP and the Connection Machine (CM-1).
- A type of parallel computer
- Single instruction: All processing units execute the same instruction at any given clock cycle
- Multiple data: Each processing unit can operate on a different data element



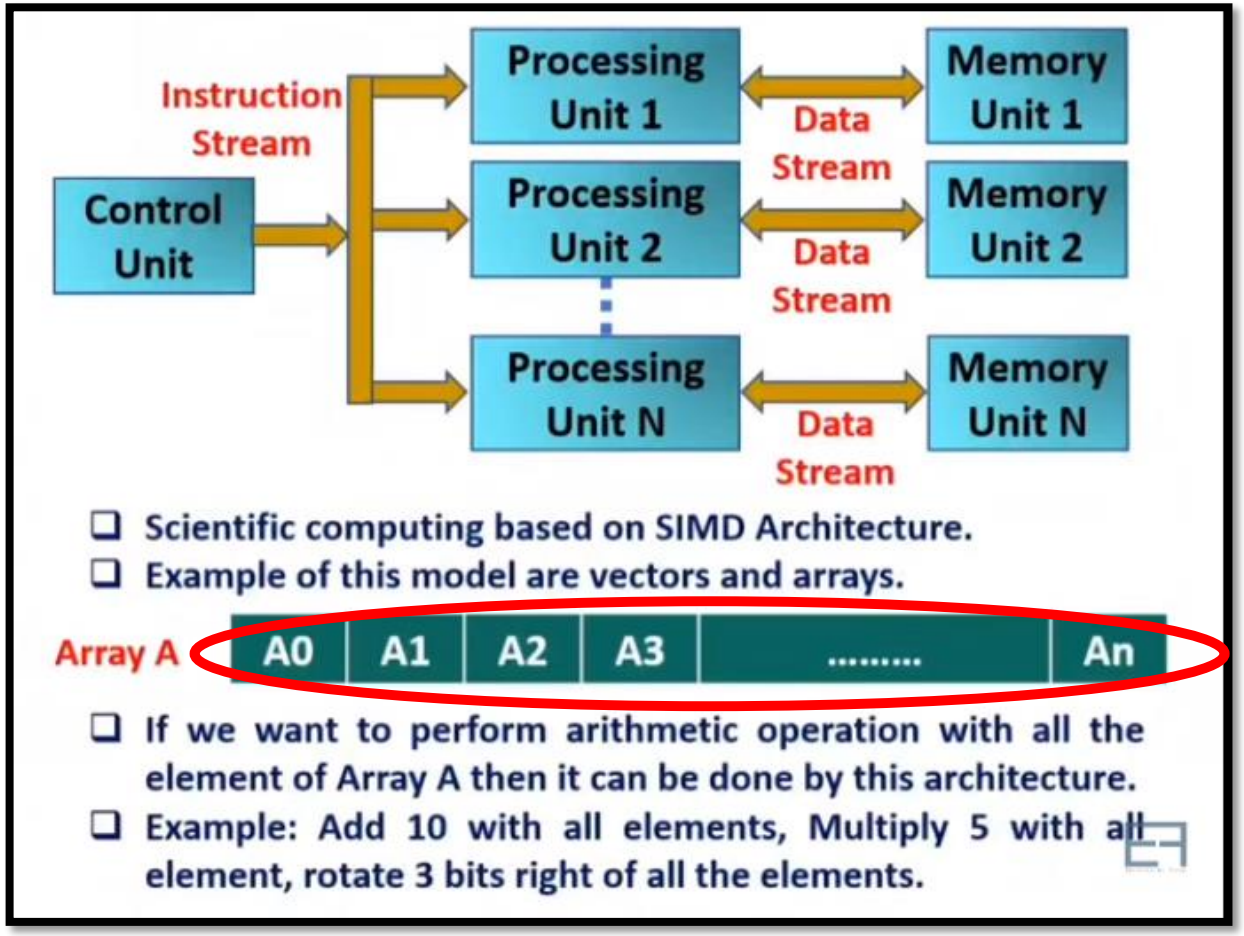
Flynn's Taxonomy - SIMD

- A SIMD system is a multiprocessor machine capable of executing the same instruction on all the CPUs but operating on different data streams.
- Machines based on a SIMD model are well suited to scientific computing since they involve lots of vector and matrix operations.
- Information can be passed to all the processing elements (PEs) organized data elements of vectors can be divided into multiple sets(N-sets for N-Processor systems) and each Processor can process one data set.



CU= Control Unit
PE= Processing Element
M= Memory
IS= Instruction Stream
DS= Data Stream
LM=Local Memory

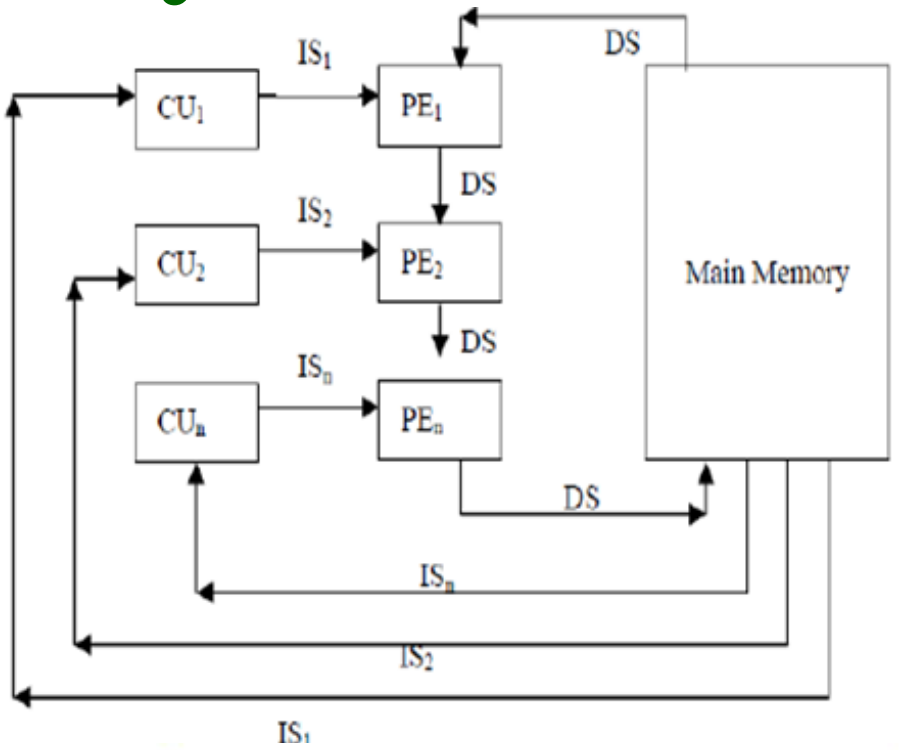
Flynn's Taxonomy - SIMD

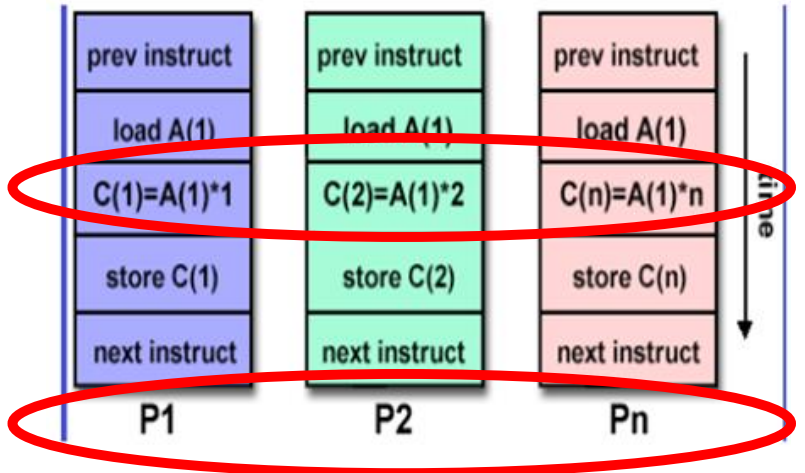


- **At one time, one instruction operates on many data**
  - Data parallel architecture–Vector architecture has similar characteristics, but achieve the parallelism with pipelining
- **Each instruction is executed on a different set of data by different processors** i.e multiple processing units by different processors i.e multiple processing units of the same type process on multiple-data streams.
- This group is dedicated to array processing machines.
- SIMD computers operate on vectors of data.

Flynn's Taxonomy - MISD

- Multiple processing elements are organised under the control of multiple control units.
- Each control unit is handling one instruction stream and processed through its corresponding processing element.
- A single data stream is fed into multiple processing units.
- Each processing unit operates on the data independently via independent instruction streams.
- But each processing element is processing only a single data stream at a time
- All processing elements are interacting with the common shared memory.
- The only known example of a computer capable of MISD operation is the C.mmp built by Carnegie-Mellon University.
- $I_s > 1$
- $D_s = 1$

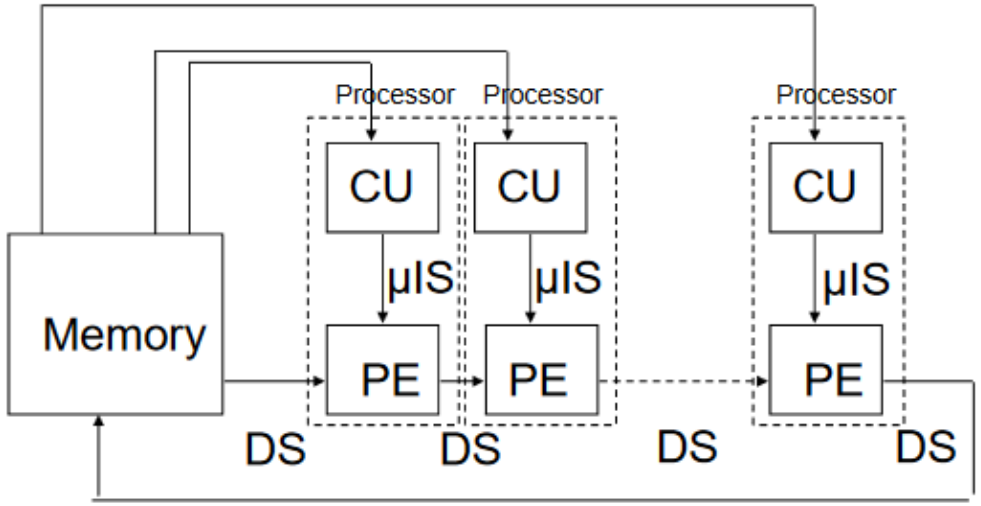




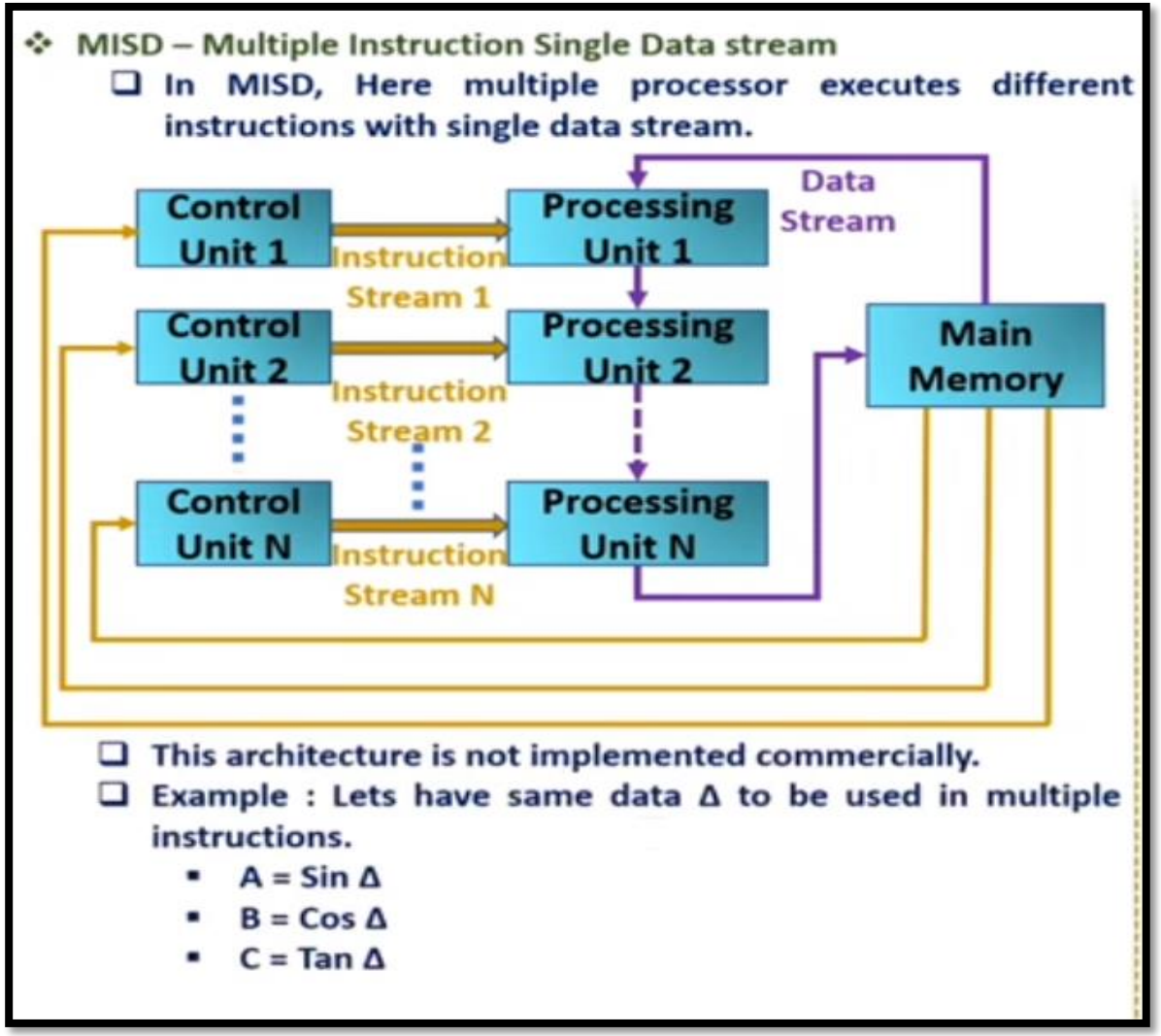
Real time computers need to be **fault tolerant** where several processors execute the same data for producing the redundant data. This is also known as **N- version programming**. All these redundant data are compared as results which should be same; otherwise faulty unit is replaced. Thus MISD machines can be applied to fault tolerant real time computers.

Flynn's Taxonomy - MISD

- An MISD computing system is a **multiprocessor machine** capable of executing **different instructions on different Processors** but all of them operating on the **same dataset**
- Each processor executes a different sequence of instructions.
- In case of MISD computers, **multiple processing units operate on one single-data stream** .
- In practice, this kind of organization **has never been used**

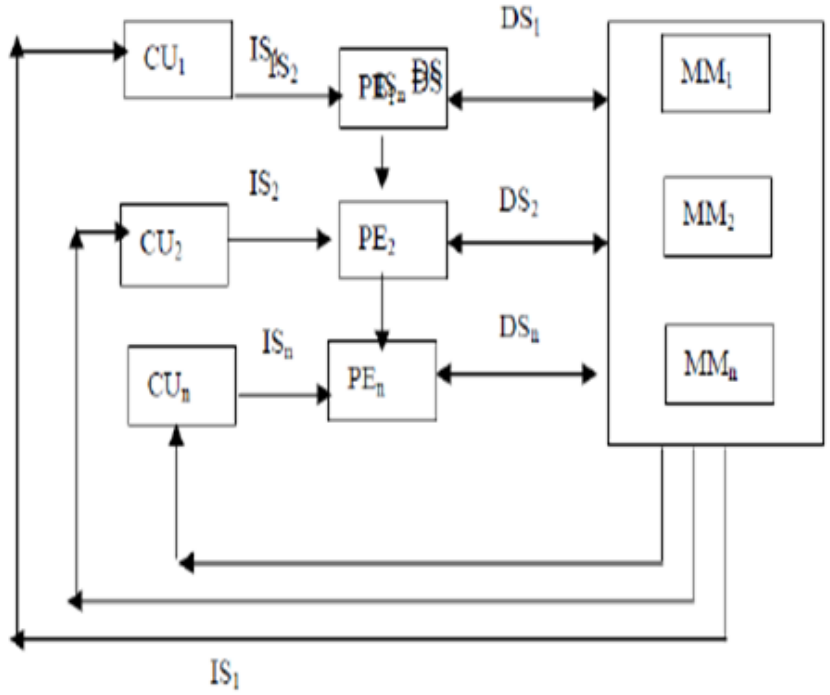


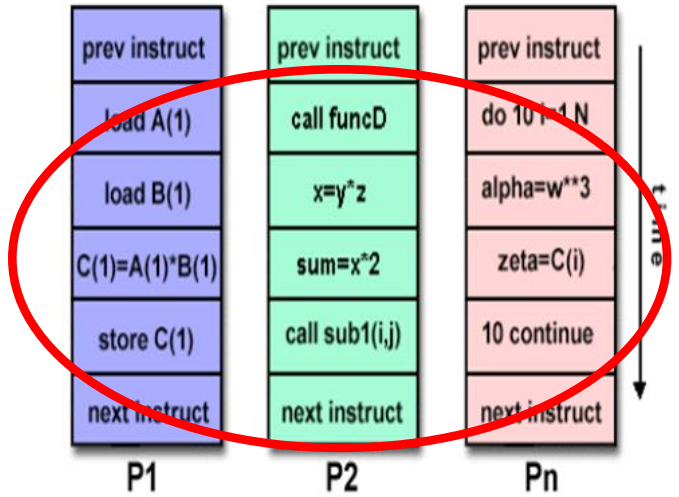
CU= Control Unit, PE= Processing Element, M= Memory



Flynn's Taxonomy - MIMD

- Multiple instruction streams operate on multiple data streams
- The processors work on their own data with their own instructions.
- Tasks executed by different processors can start or finish at different times.
- This classification actually recognizes the parallel computer.
- Examples include; C.mmp, Burroughs D825, Cray-2, S1, Cray X-MP, HEP, Pluribus, IBM 370/168 MP, Univac 1100/80, Tandem/16, IBM 3081/3084, C.m*, BBN Butterfly, Meiko Computing Surface (CS-1), FPS T/40000, iPSC.
- MIMD organization is the most popular for a parallel computer.
- In the real sense, parallel computers execute the instructions in MIMD mode.
- $I_S > 1$, $D_S > 1$





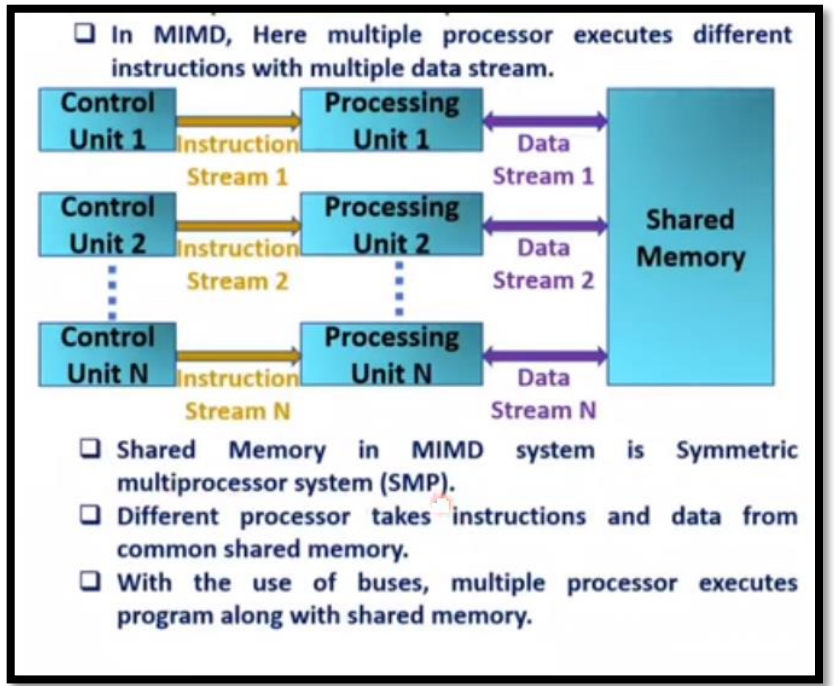
Flynn's Taxonomy - MIMD

- An MIMD system is a **multiprocessor machine** which is capable of executing **multiple instructions on multiple data sets.**
- Each Processor in the MIMD model has separate instruction and data streams; therefore machines built using this model are capable to any kind of application.
- SMPs (Symmetric Multi Processor systems), clusters, and NUMA(Non-Uniform Memory Access) systems fit into this category.
- Most common and general parallel machine
- Each processor has a separate program.
- An instruction stream is generated from each program
- Each instruction operates on different data.
- Several processing units operate on multiple-data streams.

Flynn's Taxonomy - MIMD

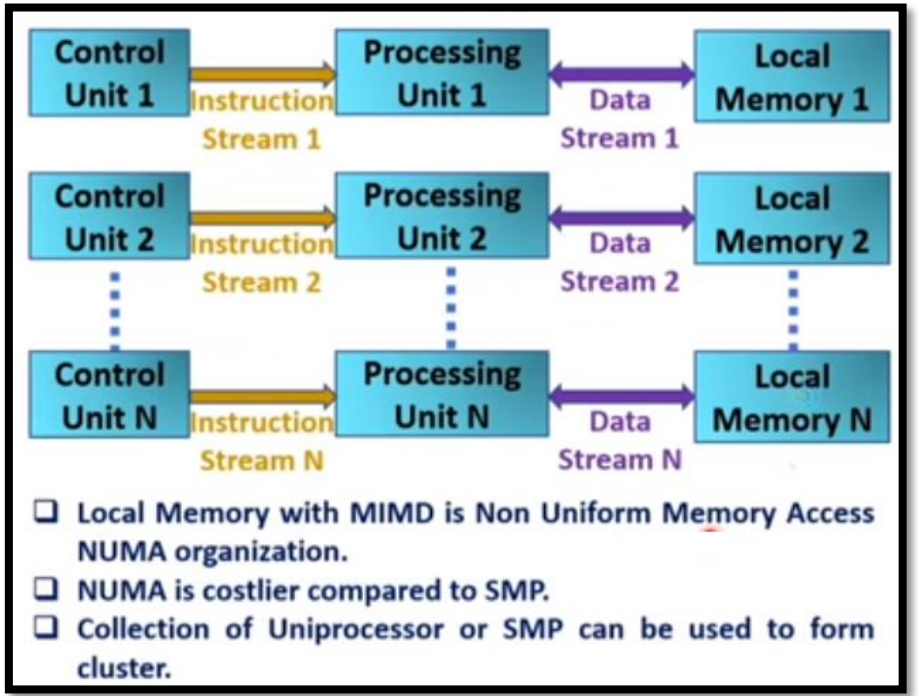
1. MIMD with shared memory

- If the processors share a common memory, then **each processor accesses programs and data stored in the shared memory.**
- The processors also communicate with each other via the shared memory.



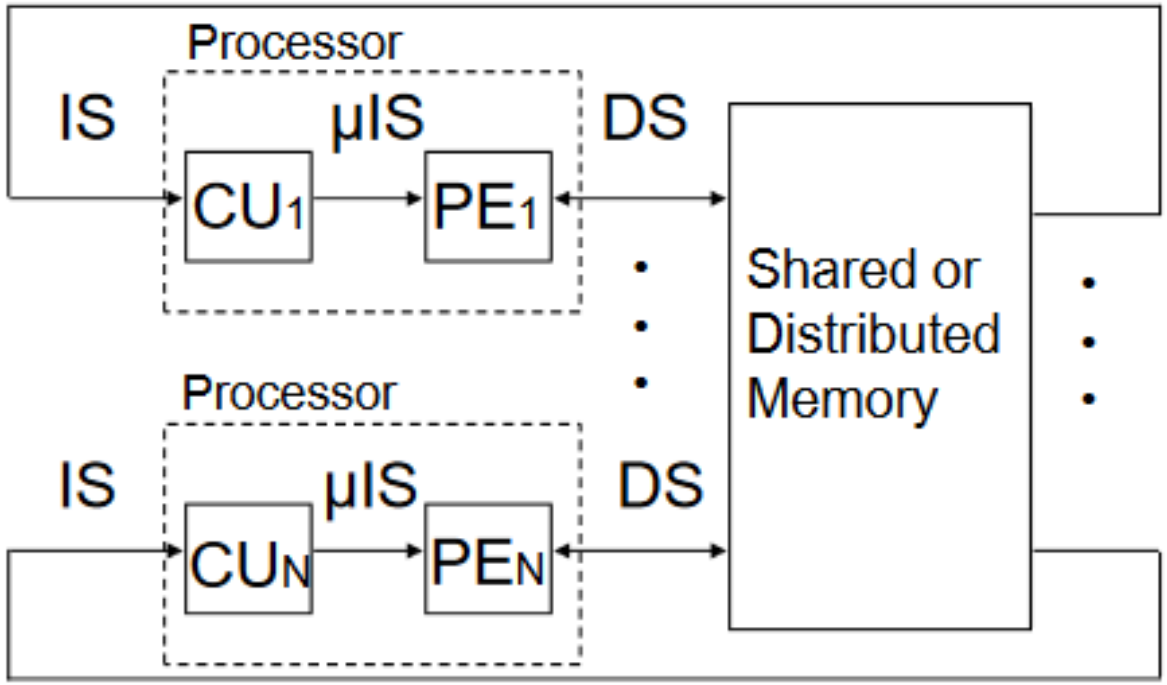
2. MIMD with Distributed memory

- An alternative to using a single shared memory is to **distribute the memory by subdividing it into a number of separate modules** with each module assigned to a different processing element



Flynn's Taxonomy - MIMD

3. MIMD with Shared or Distributed memory



CU= Control Unit, PE= Processing Element, M= Memory
IS= Instruction Stream, DS= Data Stream

BCSE205L
Computer Architecture and Organization

Module 7 – Parallelism - Pipelining

Module:7	High Performance Processors	7 Hours
Classification of models - Flynn's taxonomy of parallel machine models (SISD, SIMD, MISD, MIMD) - Pipelining: Two stages, Multi stage pipelining, Basic performance issues in pipelining, Hazards, Methods to prevent and resolve hazards and their drawbacks - Approaches to deal branches - Superscalar architecture: Limitations of scalar pipelines, superscalar versus super pipeline architecture, superscalar techniques, performance evaluation of superscalar architecture - performance evaluation of parallel processors: Amdahl's law, speed-up and efficiency.		

Parallelism

- Data Parallelism
  - **Parallelism w.r.t data**
  - **Many data items can be processed** in the same manner at the same time
  - SIMD or Vector processors
- Functional Parallelism
  - **Parallelism w.r.t modules**
  - Program has different **independent modules that can execute simultaneously**
- Instruction-level Parallelism (ILP)
  - **Parallelism w.r.t instructions**
  - Family of processor and compiler design techniques that improves performance
  - Parallel or **simultaneous execution of a sequence of instructions** of a computer program.
  - Measure of the number of instructions that can be performed during a single clock cycle

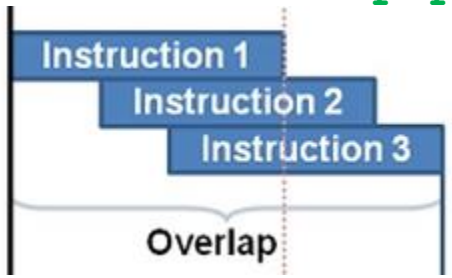
Implementations of ILP

- Pipelining
- Superscalar Architecture
  - Dependency checking on chip
  - Multiple Processing Elements (eg. ALU, Shift)
- VLIW (Very Long Instruction Word Architecture)
  - Simple hardware, Complex Compiler
- Multi processor computers

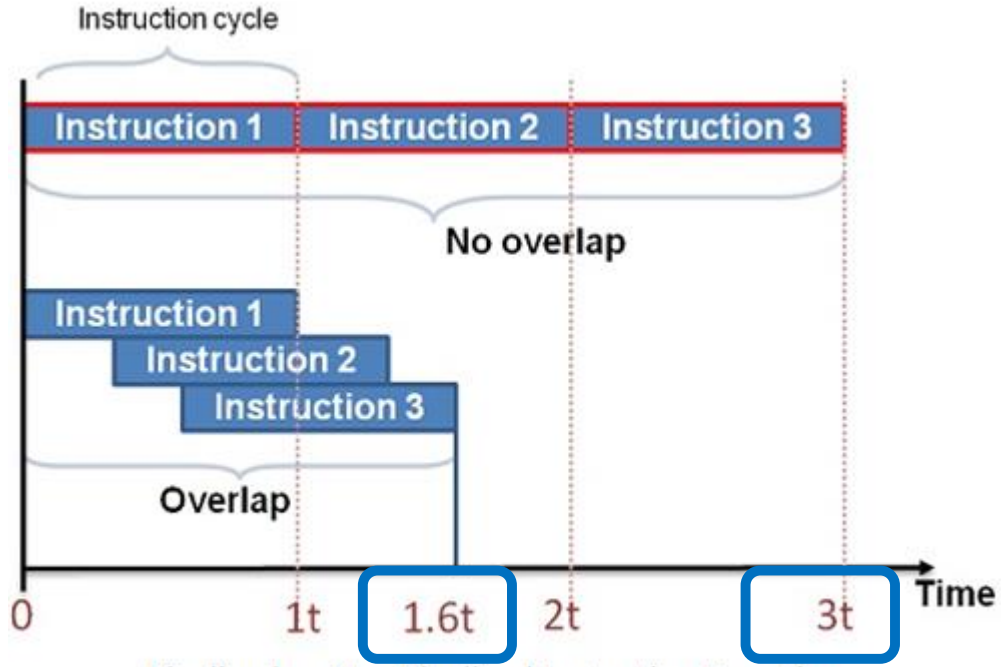
Pipelining

- Overlapping of instructions partially
- Pipelining is a process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Simultaneous execution of more than one instruction takes place in a pipelined processor.
- Technique of decomposing a sequential process into sub-operations, with each sub-operation being executed in a dedicated segment that operates concurrently with all other segments.
- Pipelining improves instruction throughput rather than individual instruction execution time

Non-pipelined Execution



Pipelined Execution



Pipelined vs Non-Pipelined Instruction Execution

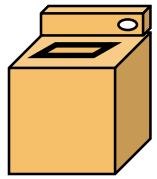
Pipelining

Laundry Example

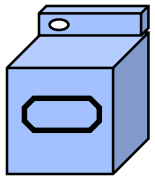
Ann, **B**rian, **C**athy, **D**ave

Each has one load of clothes to

wash, dry, fold.



washer
30 mins

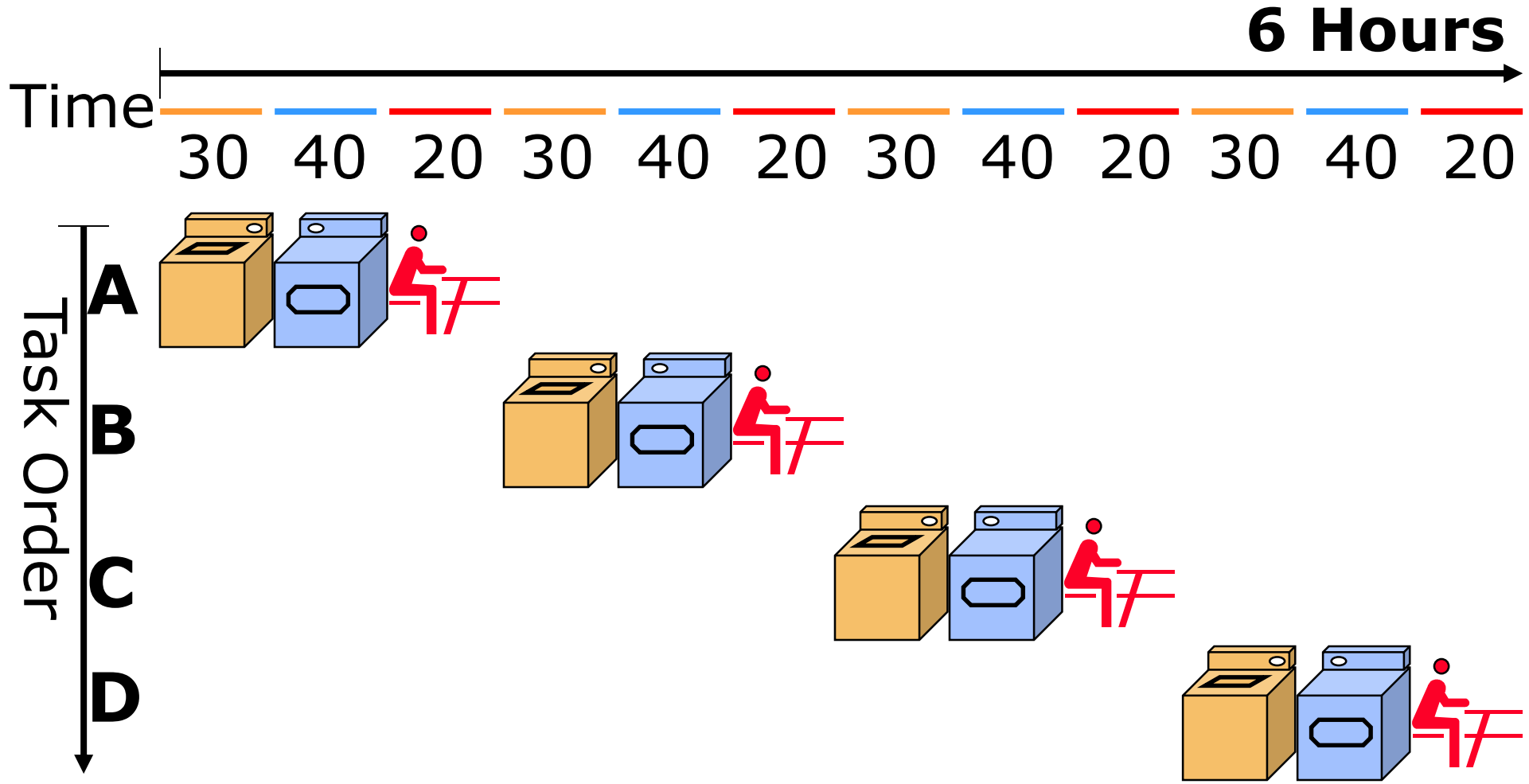


dryer
40 mins

folder
20 mins

Pipelining

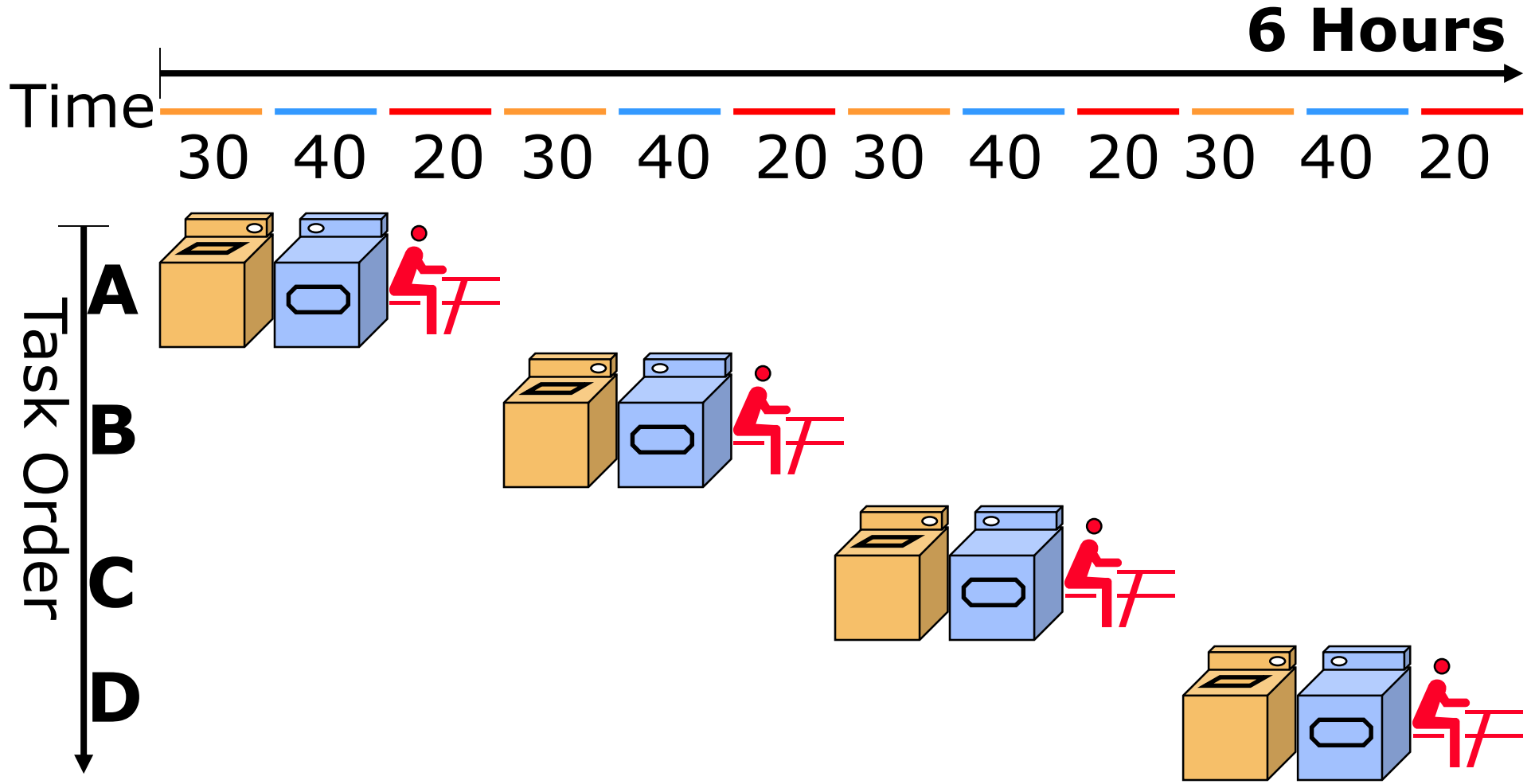
Sequential Laundry



What would you do?

Pipelining

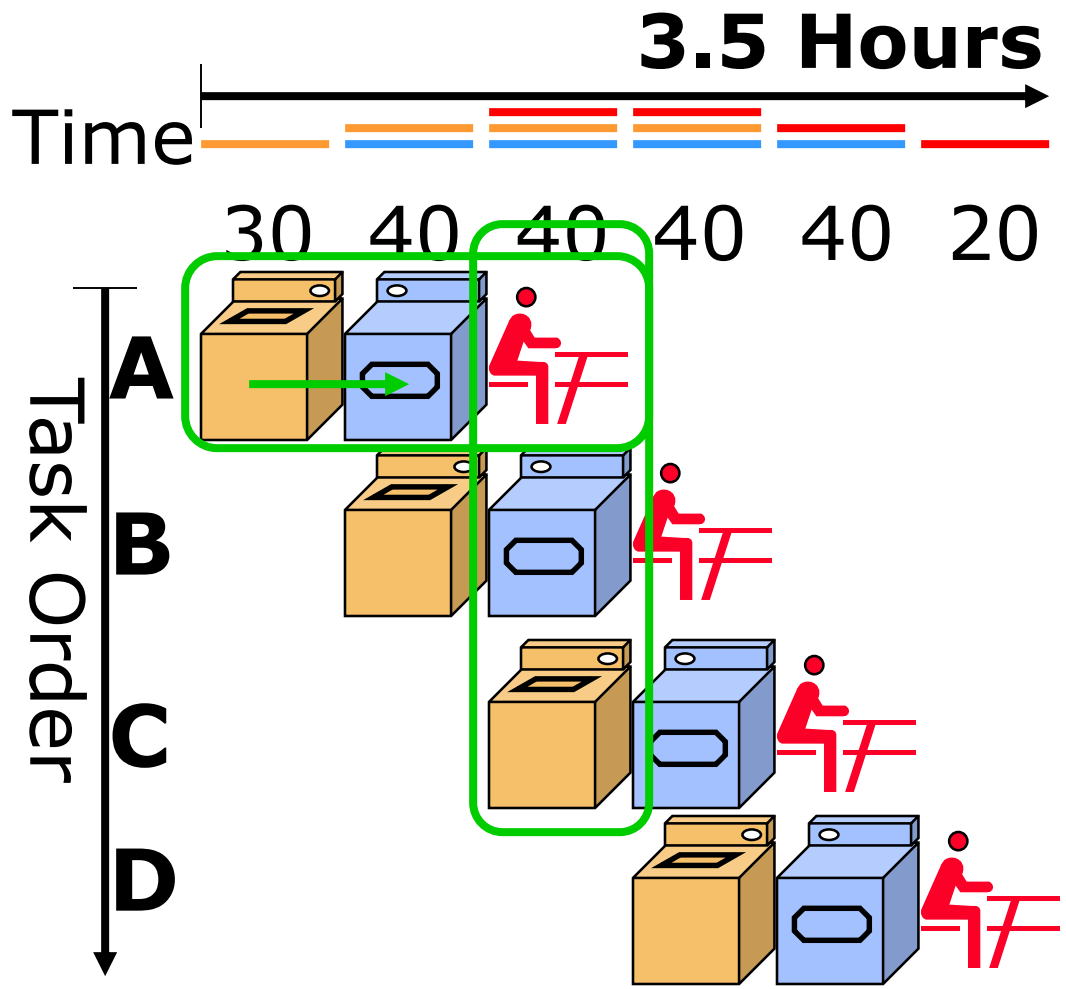
Sequential Laundry



What would you do?

Pipelining

Pipelined Laundry



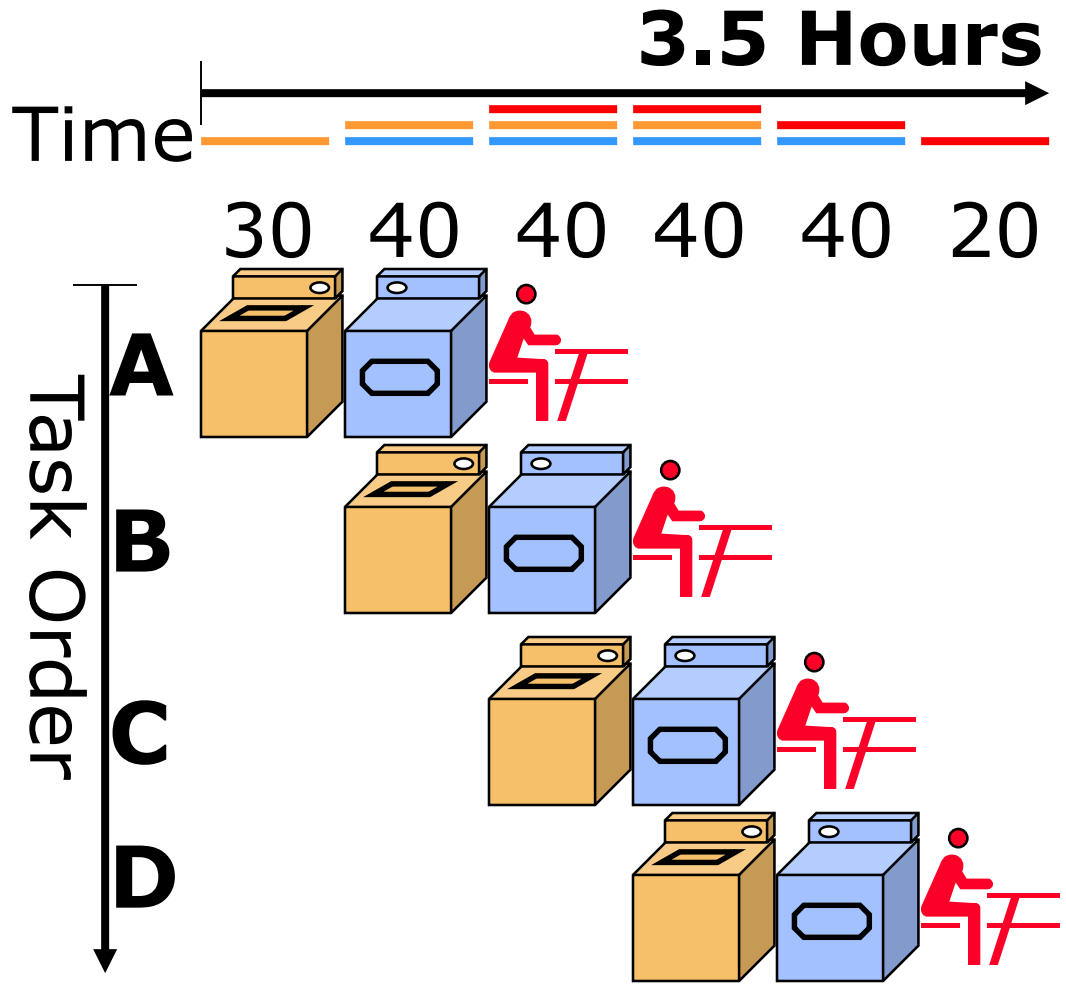
Observations

- A task has a series of stages;
- Stage dependency:

  e.g., wash before dry;
- Multi tasks with overlapping stages;
- Simultaneously use diff resources to speed up;
- Slowest stage determines the finish time;

Pipelining

Pipelined Laundry



Observations

- No speed up for individual task;

  e.g., A still takes 30+40+20=90

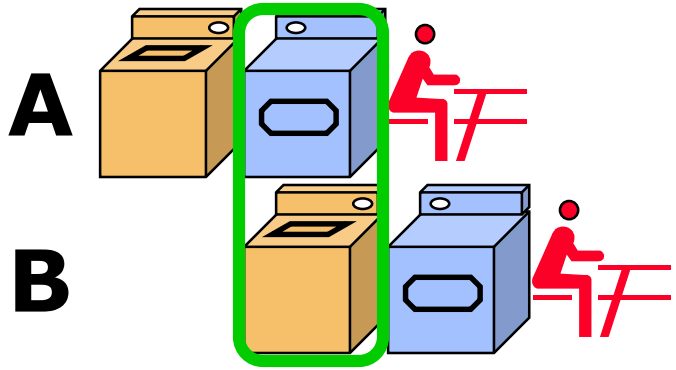
- But speed up for average task execution time;

  e.g., 3.5*60/4=52.5 < 30+40+20=90

Pipelining

- An implementation technique whereby multiple instructions are overlapped in execution.

  e.g., B wash while A dry



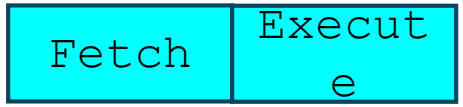
- **Essence:** Start executing one instruction before completing the previous one.
- **Significance:** Make fast CPUs.

Pipelining

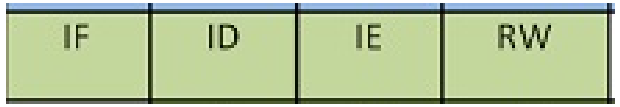
- A **Pipelining** is a series of stages, where some work is done at each stage in parallel.
- The stages are connected one to the next to form a pipe - instructions enter at one end, progress through the stages, and exit at the other end.

Instruction Pipelining

Two Stage Pipeline



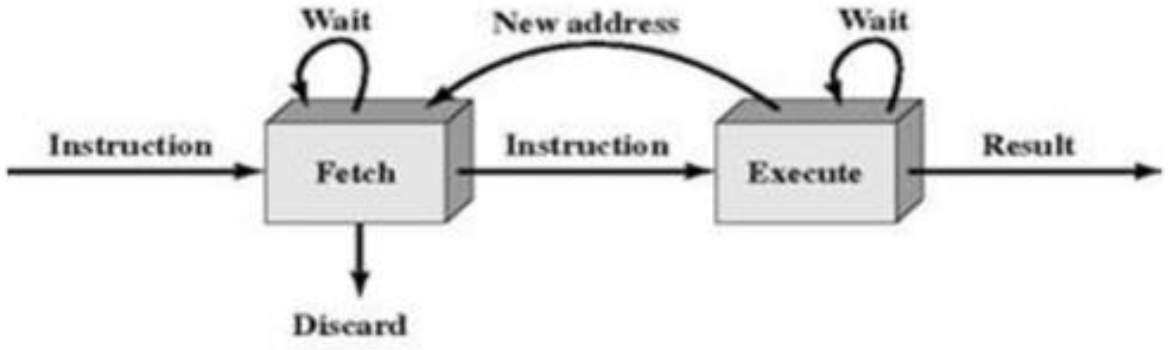
Four Stage Pipeline

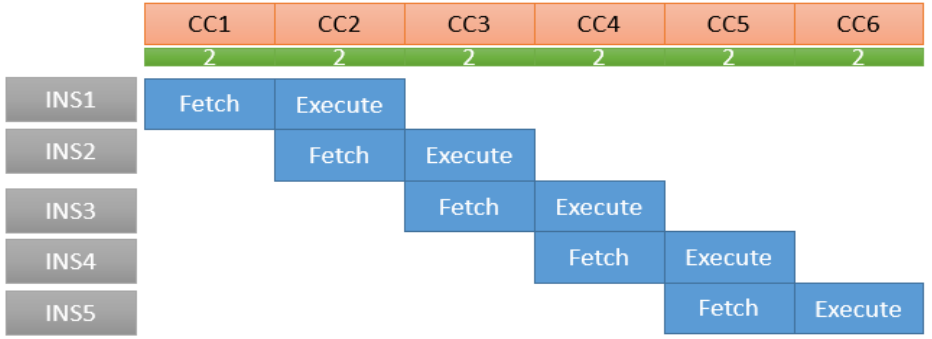


Six Stage Pipeline

Pipelining
– Two Stage Instruction Pipelining

- The pipeline has two independent stages.
- First stage - fetches an instruction and buffers it. When the second stage is free, the first stage passes it the buffered instruction.
- Second stage - executing the instruction, the first stage takes advantage of any unused memory cycles to fetch and buffer the next instruction. This is called instruction prefetch or fetch overlap.





Pipelining
– Two Stage Instruction Pipelining

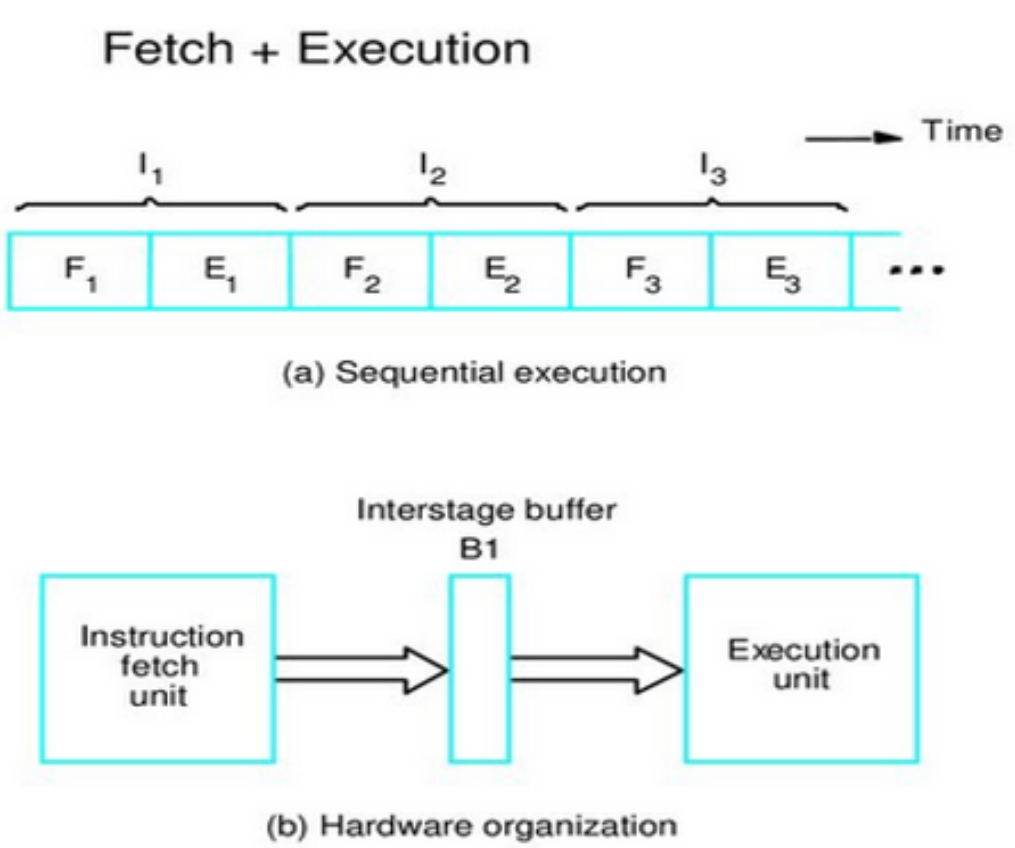
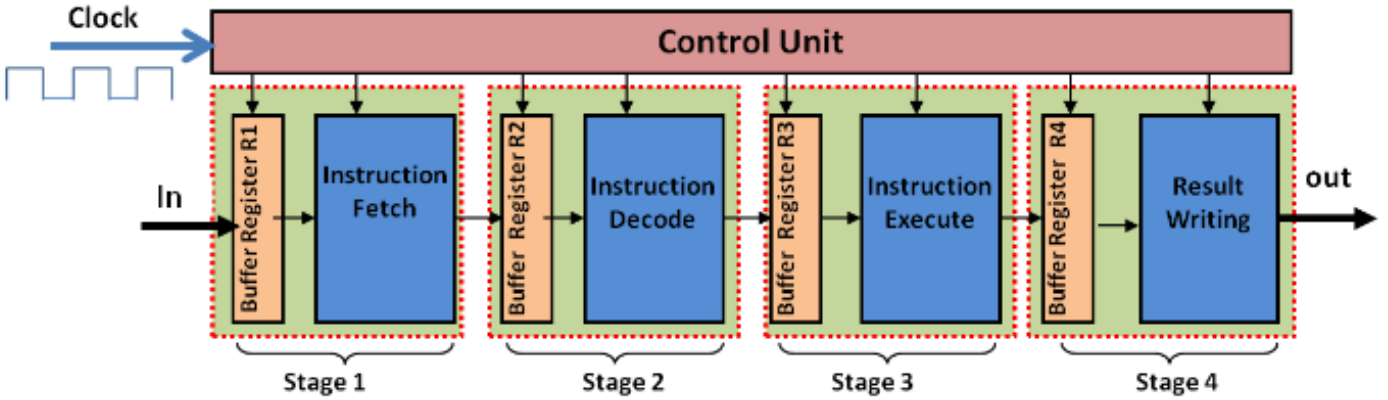


Figure 8.1. Basic idea of instruction pipelining.

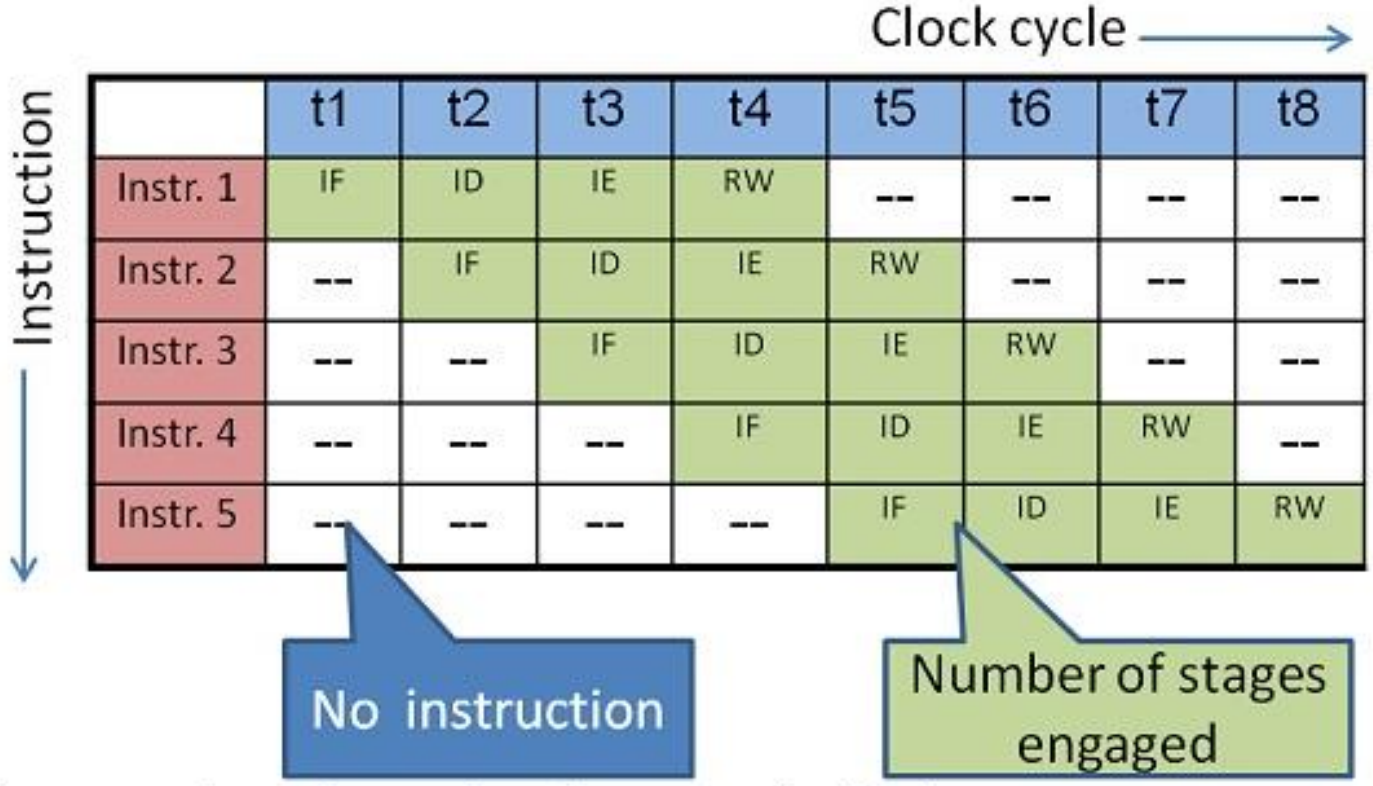
Pipelining
– Four Stage Instruction Pipelining

Four stages:



- **Instruction Fetch (IF)** from memory
- **Instruction Decode (ID)** in CPU
- **Instruction Execution (IE)** in ALU
- **Result Writing (RW)** in memory or Register.
- Since there are four stages, all the instructions pass through the four stages to complete the instruction execution.

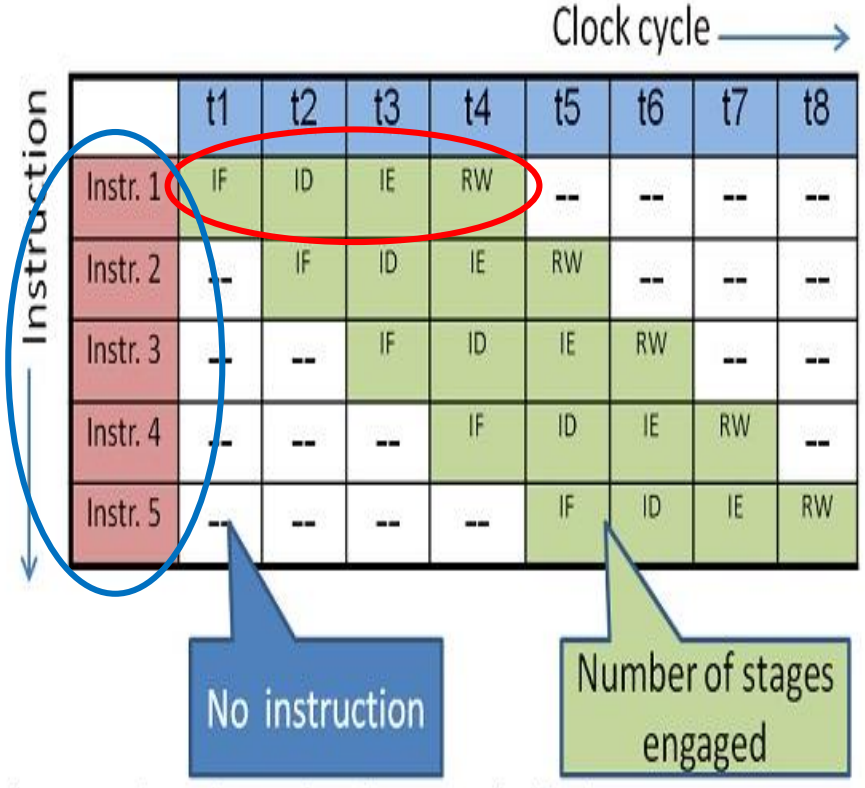
Pipelining – Four Stage Instruction Pipelining

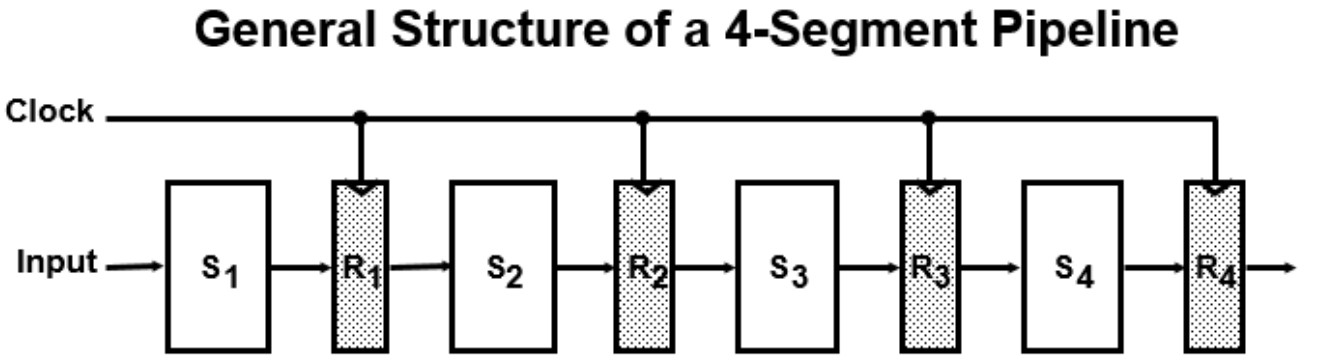


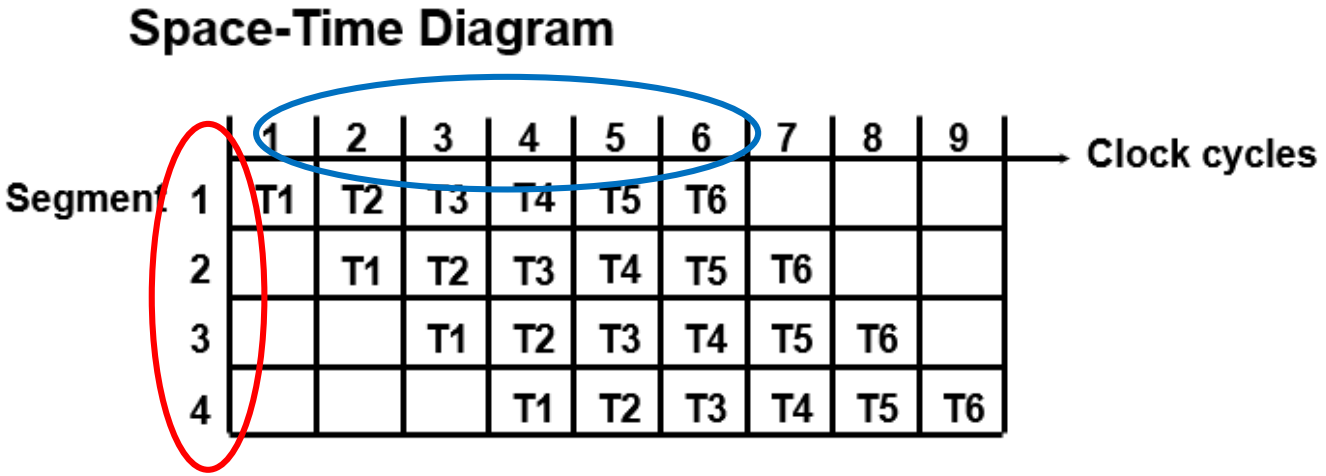
Performance:

- In 8 clock cycles, 5 instructions have got executed in a four-stage pipelined design.
- The same would have taken 20 (5 instructions x 4 cycles for each) clock cycles in a non pipelined architecture.
- The **performance improvement depends on the number of stages** in the design.

Pipelining
– Four Stage Instruction Pipelining







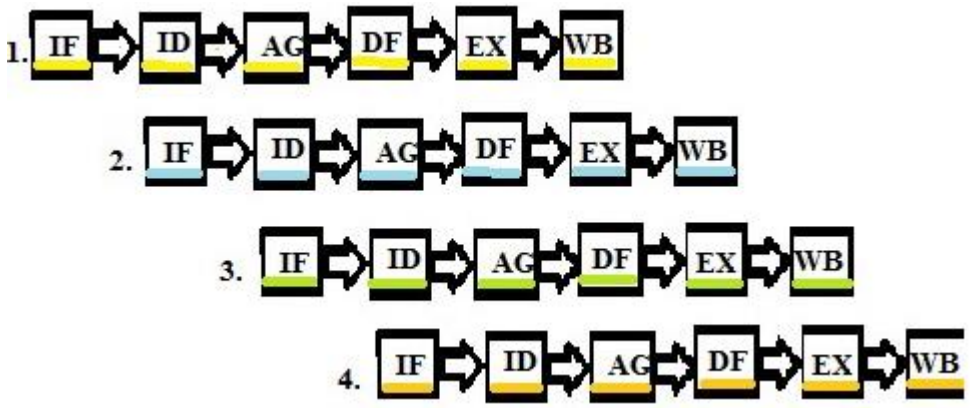
Pipelining
– Six Stage Instruction Pipelining

- A typical instruction cycle can be split into many sub cycles like Fetch instruction, Decode instruction, Execute and Store.
- The instruction cycle and the corresponding sub cycles are performed for each instruction. These sub cycles for different instructions can thus be interleaved or in other words these sub cycles of many instructions can be carried out simultaneously, resulting in **reduced overall execution time**. This is called instruction pipelining.
- The **more are the stages in the pipeline, the more the throughput is of the CPU.**
- If the **instruction processing is split into six phases**, the pipelined CPU will have six different stages for the execution of the sub phases.

Pipelining
– Six Stage Instruction Pipelining

Stages

- Fetch Instruction (FI)
- Decode Instruction ((DI)
- Calculate Operand (CO)
- Fetch Operands (FO)
- Execute Instruction (EI)
- Write Operand (WO)



- **FI:** Instructions are fetched from the memory into a temporary buffer before it gets executed.
- **DI:** The instruction is decoded by the CPU so that the necessary op codes and operands can be determined. **(Instruction Decode)**
- **CO:** Based on the addressing scheme used, either operands are directly provided in the instruction or the effective address has to be calculated. **(Address Generator)**
- **FO:** Once the address is calculated, the operands need to be fetched from the address that was calculated. This is done in this phase. **(Data Fetch)**
- **EI:** The instruction can now be executed.
- **WO:** Once the instruction is executed, the result from the execution needs to be stored or written back in the memory. **(Write Back)**

Pipelining – Six Stage Instruction Pipelining

The timing diagram of a six stage instruction pipeline is shown

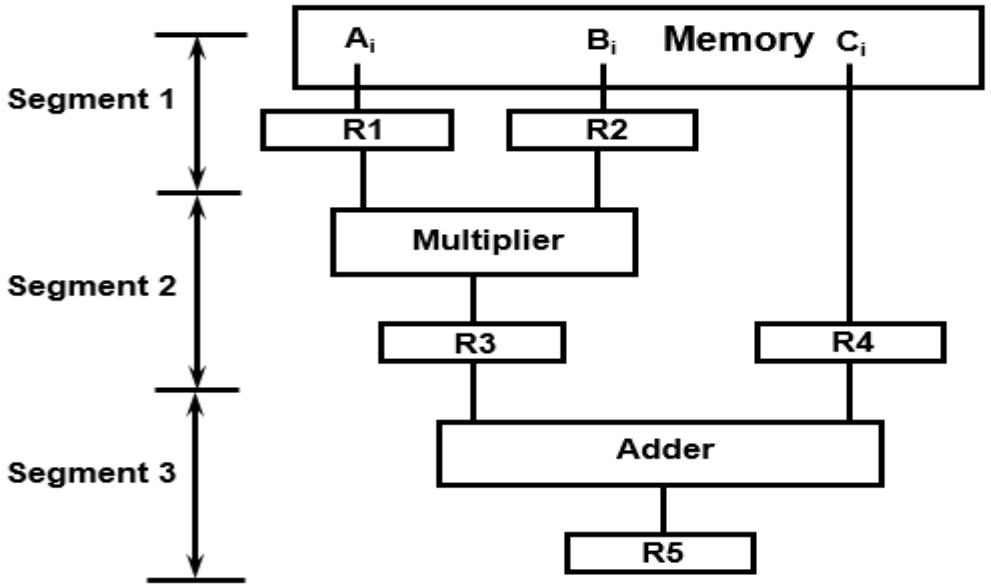
Clock	1	2	3	4	5	6	7	8
I_1	FI	DI	CO	FO	EI	WO		
I_2		FI	DI	CO	FO	EI	WO	
I_3			FI	DI	CO	FO	EI	WO

- In case the time required by each of the sub phase is not same appropriate delays need to be introduced.
- From this timing diagram it is clear that the **total execution time of 3 instructions in this 6 stages pipeline is 8-time units.**
- The first instruction gets completed after 6 time unit, and thereafter in each time unit it completes one instruction.
- **Without pipeline**, the **total time required to complete 3 instructions would have been 18 (6*3) time units.** Therefore, there is a speed up in pipeline processing and the speed up is related to the number of stages.

Pipelining

A technique of decomposing a sequential process into suboperations, with each subprocess being executed in a partial dedicated segment that operates concurrently with all other segments.

$A_i * B_i + C_i$ for i = 1, 2, 3, ... , 7



$R1 \leftarrow A_i$, $R2 \leftarrow B_i$	Load A_i and B_i
$R3 \leftarrow R1 * R2$, $R4 \leftarrow C_i$	Multiply and load C_i
$R5 \leftarrow R3 + R4$	Add

OPERATIONS IN EACH PIPELINE STAGE

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A1	B1			
2	A2	B2	A1 * B1	C1	
3	A3	B3	A2 * B2	C2	A1 * B1 + C1
4	A4	B4	A3 * B3	C3	A2 * B2 + C2
5	A5	B5	A4 * B4	C4	A3 * B3 + C3
6	A6	B6	A5 * B5	C5	A4 * B4 + C4
7	A7	B7	A6 * B6	C6	A5 * B5 + C5
8			A7 * B7	C7	A6 * B6 + C6
9					A7 * B7 + C7

Speed Up and Efficiency

- **For a pipeline processor:**
  - ***k-stage*** pipeline processes with a clock cycle time t_p is used to execute *n* tasks.
  - The time required for the first task T_1 to complete the operation = **$k*t_p$**
    (if k segments in the pipe)
  - The **time required** to complete (n-1) tasks = **$(n-1)*t_p$**
  - Therefore to complete n tasks using a k-segment pipeline requires = **$k + (n-1)*t_p$ clock cycles.**
- **For the non-pipelined processor :**
  - Time to complete each task = t_n
  - Total time required to complete n tasks=$n*t_n$
- **Speed up** = Time reqd. by non pipelining processing/Time reqd. by pipelining processing

$$S = \frac{T_1}{T_K} = \frac{nt_n}{(k + (n-1))t_p}$$

Speed Up and Efficiency

- Latency of pipeline = no of stages * cycle time
- Pipeline Cycle Time = Maximum delay due to any stage + Delay due to its register (Latch latency)
- Speed up = non pipelining processing/pipelining processing

- **For a pipeline processor:**
  - *k-stage* pipeline processes with a clock cycle time t_p is used to execute *n* tasks.
  - The time required for the first task T_1 to complete the operation = $k*t_p$ (if k segments in the pipe)
  - The time required to complete (n-1) tasks = $(n-1)*t_p$
  - Therefore to complete n tasks using a k-segment pipeline requires =k + (n-1) clock cycles.
- **For the non-pipelined processor :**
  - Time to complete each task = t_n
  - Total time required to complete n tasks=$n*t_n$
  - Speed up = non pipelining processing/pipelining processing

$$S = \frac{T_1}{T_K} = \frac{nt_n}{(k + (n-1))t_p}$$

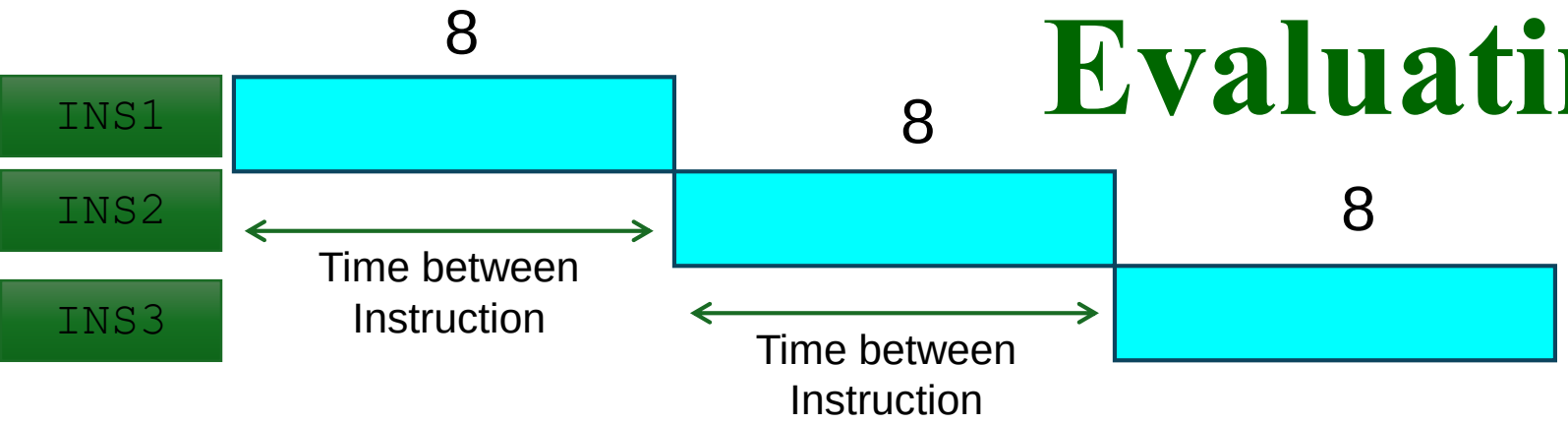
Speed Up and Efficiency

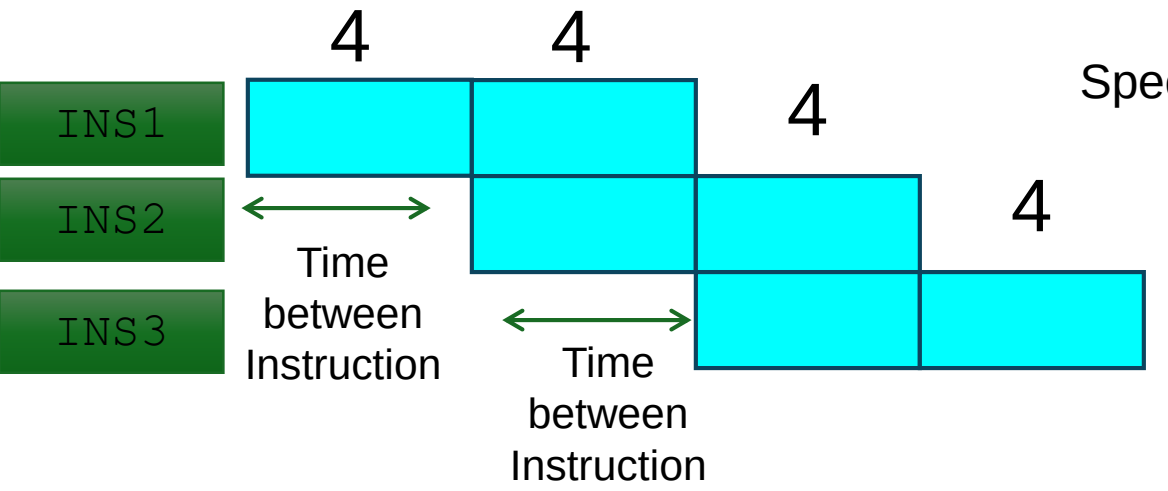
- As the number of tasks increases, n becomes much larger than k-1, and approaches the value of n. *(i.e., k+n-1 becomes n at some point)*
- Under this condition, speed up becomes

$$S = t_n / t_p$$

- Assume the time taken for pipeline and non pipeline circuits are same then $t_n = k * t_p$
- Speed up reduces to $S = (k * t_p)/t_p = k$
- This shows that the theoretical **maximum speedup** that a pipeline can provide is k, where k is the number of segments in the pipeline.

Evaluating Speed Up…





$$\text{Speed up} = \frac{\text{Time Between the instruction Unpipelined}}{\text{Time Between the instruction pipelined}} = \frac{8}{4} = 2$$

? What is your observation from speedup factor Vs No. of Stages

Pipelining – Problem 1

- 4-stage pipeline
- subopertion in each stage; t_p = 20nS
- 100 tasks to be executed
- 1 task in non-pipelined system; 20*4 = 80nS

Pipelining – Problem 1

- 4-stage pipeline
- subopertion in each stage; t_p = 20nS
- 100 tasks to be executed
- 1 task in non-pipelined system; 20*4 = 80nS

Pipelined System

$(k + n - 1)*t_p = (4 + 99) * 20 = 2060nS$

Non-Pipelined System

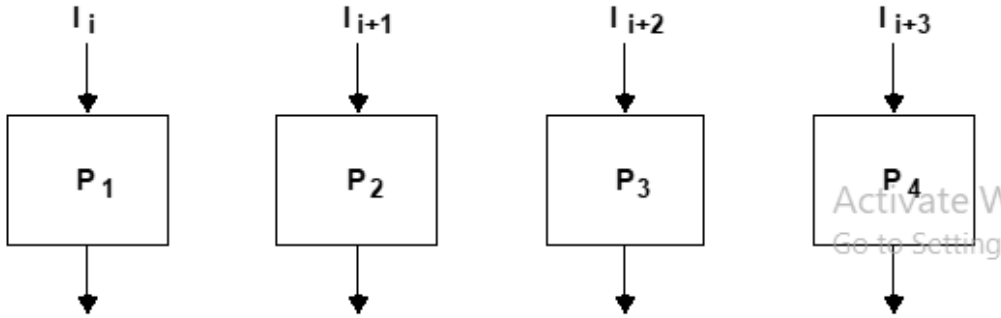
$n*k*t_p = 100 * 80 = 8000nS$

Speedup

$S_k = 8000 / 2060 = 3.88$

4-Stage Pipeline is basically identical to the system with 4 identical function units

Multiple Functional Units



Evaluating Speed Up… Five-Stage Pipeline

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Instruction *i*	IF	ID	EX	MEM	WB				
Instruction *i* + 1		IF	ID	EX	MEM	WB			
Instruction *i* + 2			IF	ID	EX	MEM	WB		
Instruction *i* + 3				IF	ID	EX	MEM	WB	
Instruction *i* + 4					IF	ID	EX	MEM	WB

Simply start a new instruction on each clock cycle;

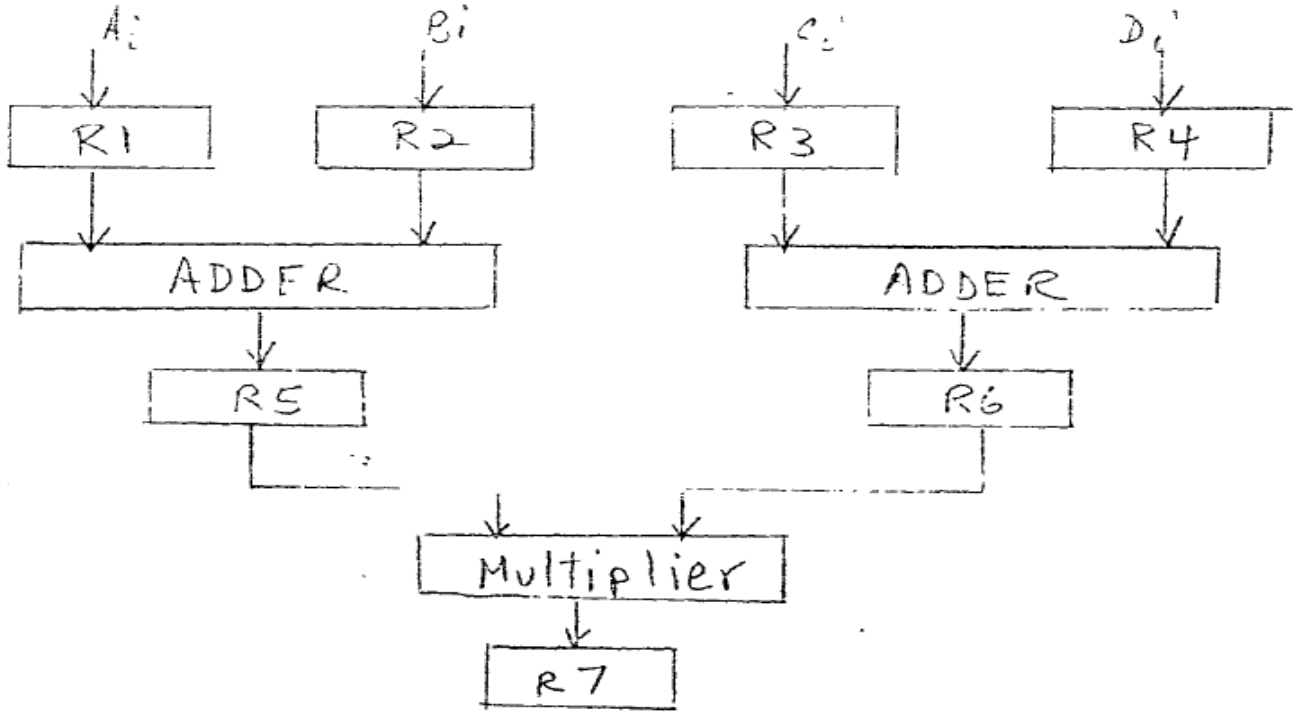
Hence, Speedup = 5.

Pipelining – Problem 2

In certain scientific computations it is necessary to perform the arithmetic operation $(A_i + B_i)(C_i + D_i)$ with a stream of numbers. Specify a pipeline configuration to carry out this task. List the contents of all registers in the pipeline for $i = 1$ through 6.

Pipelining – Problem 2

In certain scientific computations it is necessary to perform the arithmetic operation $(A_i + B_i)(C_i + D_i)$ with a stream of numbers. Specify a pipeline configuration to carry out this task. List the contents of all registers in the pipeline for $i = 1$ through 6.



Pipelining – Problem 3

Draw a space-time diagram for a six-segment pipeline showing the time it takes to process eight tasks.

Pipelining – Problem 3

Draw a space-time diagram for a six-segment pipeline showing the time it takes to process eight tasks.

Segment	1	2	3	4	5	6	7	8	9	10	11	12	13
1	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8					
2		T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8				
3			T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8			
4				T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8		
5					T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	
6						T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8

k + (n-1) clock cycles

$(k-n-1)t_p = 6+8-1 = 13$ cycles

Pipelining – Problem 4

Determine the number of clock cycles that it takes to process 200 tasks in a six-segment pipeline.

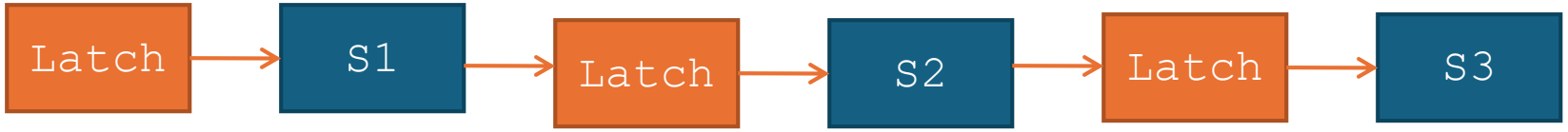
Pipelining – Problem 4

Determine the number of clock cycles that it takes to process 200 tasks in a six-segment pipeline.

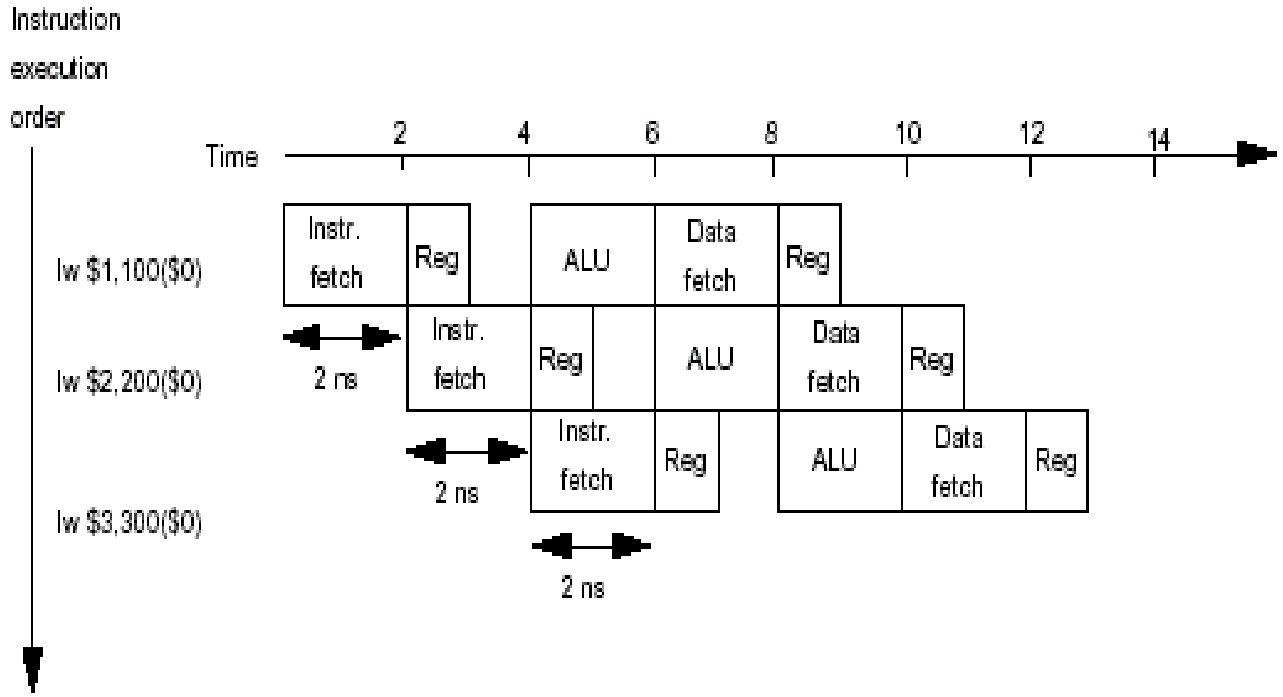
$k = 6$ segment
$n = 200$ tasks

$(k + n - 1) = 6 + 200 - 1 = 205$ cycles

How to Design a PIPE…



- Practically, it is difficult to divide instruction process into uniform stages
- Duration of stage equal to largest stage in the instruction process
- All smaller stages are associated with latch to allow delay



Pipelining Lessons…

- Pipelining - doesn't help **latency/ Turnaround** of single task

  - it helps **throughput** of entire workload

  - **latency/ Turnaround:** Complete single task in the smallest amount of time
  - **Throughput:** Complete the most tasks in a fixed amount of time

- Pipeline rate - limited by **slowest** pipeline stage
- Potential speedup = Number of pipe stages
  - Unbalanced lengths of pipe stages **reduces** speedup
  - Time to "**fill**" pipeline and time to "**drain**" it reduces speedup

Pipelining – Adv & Disadv

Advantages:

- More efficient use of processor
- Quicker time of execution of large number of instructions

Disadvantages:

- Pipelining involves adding hardware to the chip
- Inability to continuously run the pipeline at full speed because of pipeline hazards which disrupt the smooth execution of the pipeline.

Basic Performance Issues in Pipelining

- **Data hazards:** When two instructions in a program are to be executed in sequence and both access a particular memory or register operand.
  - For example, if instruction A writes to a register that instruction B reads from, and instruction B is in an earlier stage than instruction A.
- **Branching:** Branch instructions can be problematic in a pipeline if a branch is conditional on the results of an instruction that has not yet completed its path through the pipeline.
- **Timing variations:** The pipeline cannot take the same amount of time for all the stages.

Pipelining Hazards

- **Hazard** – Any condition that make pipeline to ***stall***.
- Stalling
  - A delay in instruction processing.
  - Itself can be used to resolve the hazard.
- Types of Hazards
  - Structural Hazard
  - Data Hazard
  - Instructional/Control Hazard

Pipelining Hazards

- **Data hazards (Data Dependency Conflicts)** - An instruction scheduled to be executed in the pipeline requires the result of a previous instruction, which is not yet available.
- occur when there are dependencies between instructions, that is, the output of one instruction is an input to another.

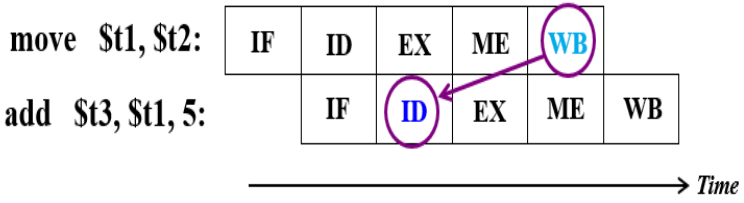
Data Dependency

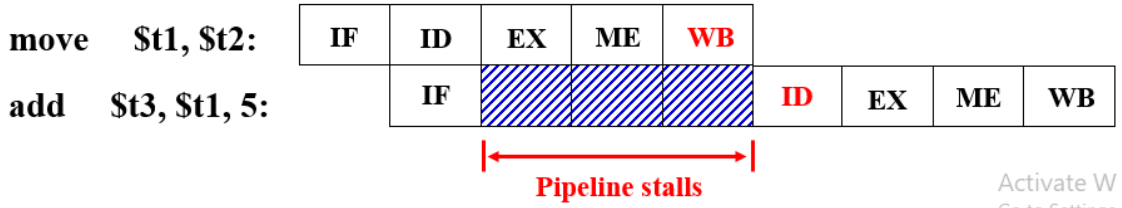
The same registers are used in more than one instruction

```
move $t1, $t2     // t1 ← t2
add  $t3, $t1, 5  // t3 ← t1 + 5
```

Resolve data dependency by stall

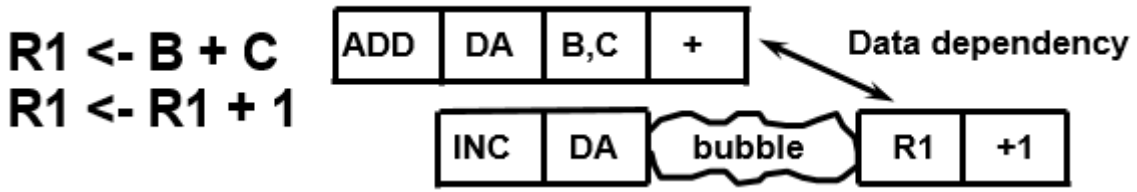
This can not be done

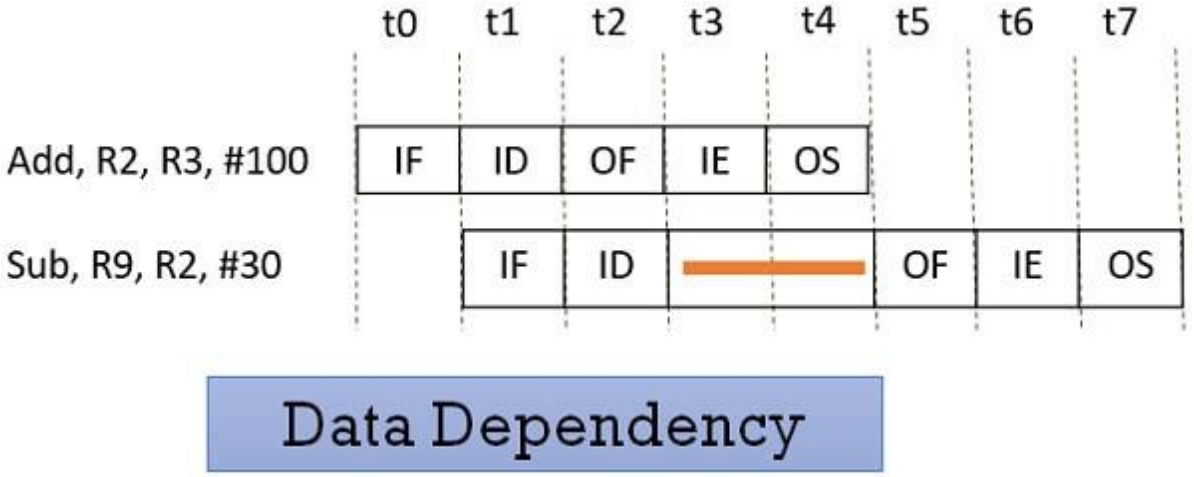




Pipelining Hazards

- **Data hazards (Data Dependency Conflicts)**





- **Categories of Data Hazards:**
  - data Read After Write hazards (RAW)
  - data Write After Read hazards (WAR)
  - data Write After Write hazards (WAW).

Pipelining Hazards

- **Categories of Data Hazards:**
  - data Read After Write hazards (RAW)

```
I: add r1,r2,r3
J: sub r4,r1,r3
```

    - Also known as a true dependency - occurs when an instruction depends on the result of a previous instruction.

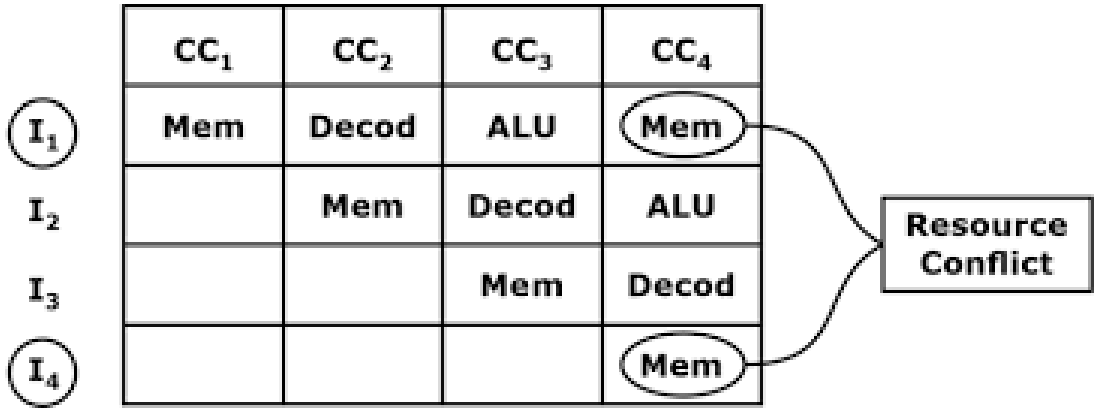
```
i1. R4 <- R1 + R5
i2. R5 <- R1 + R2
```

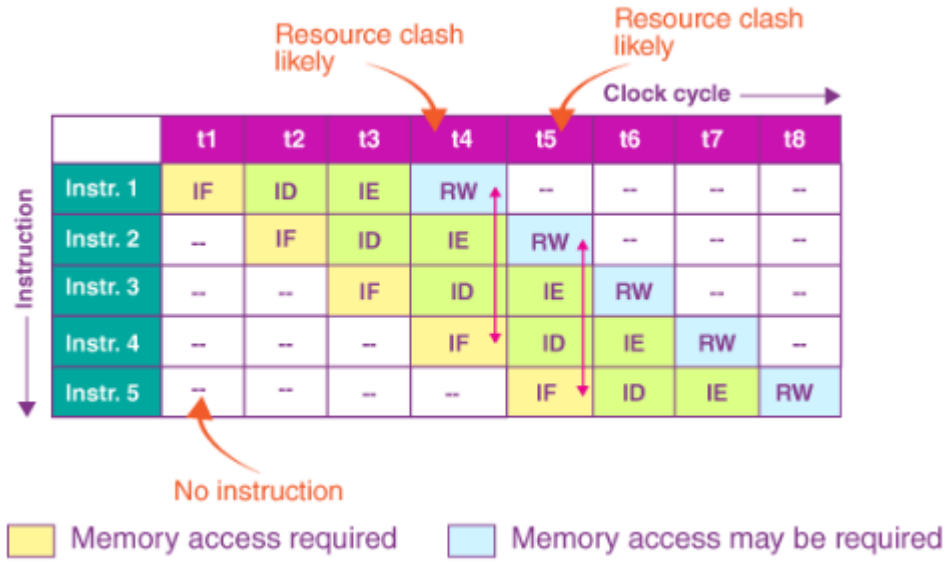
  - data Write After Read hazards (WAR)
    - Also known as anti-dependency - occurs when an instruction depends on the reading of a value before that value is overwritten by a previous instruction.
  - data Write After Write hazards (WAW)
    - Also known as output-dependency - occurs when a value is written by an instruction before the previous instruction writes that value.

```
I: sub r1,r4,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

Pipelining Hazards

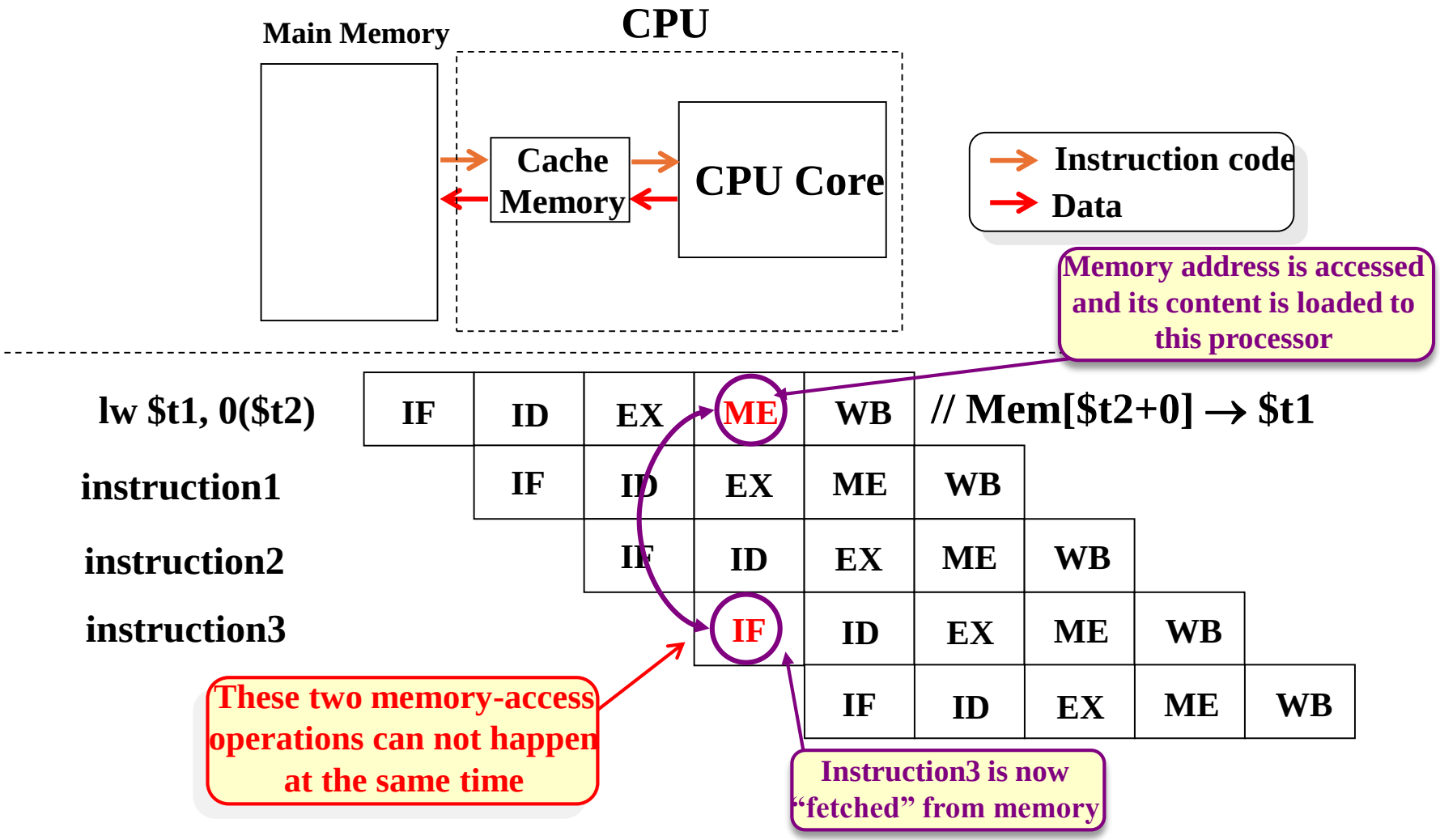
- **Structural hazards(Resource Conflicts)** - Hardware Resources required by the instructions in simultaneous overlapped execution cannot be met.
- These occur when the same hardware resource is desired by multiple instructions at the same time.





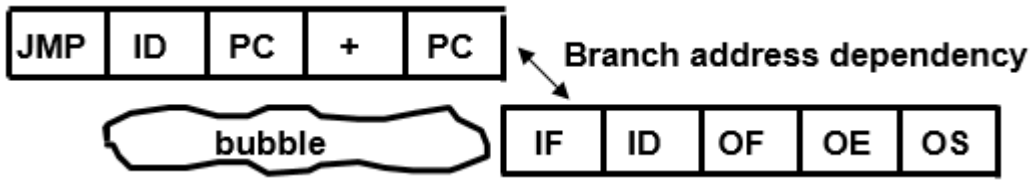
Pipelining Hazards

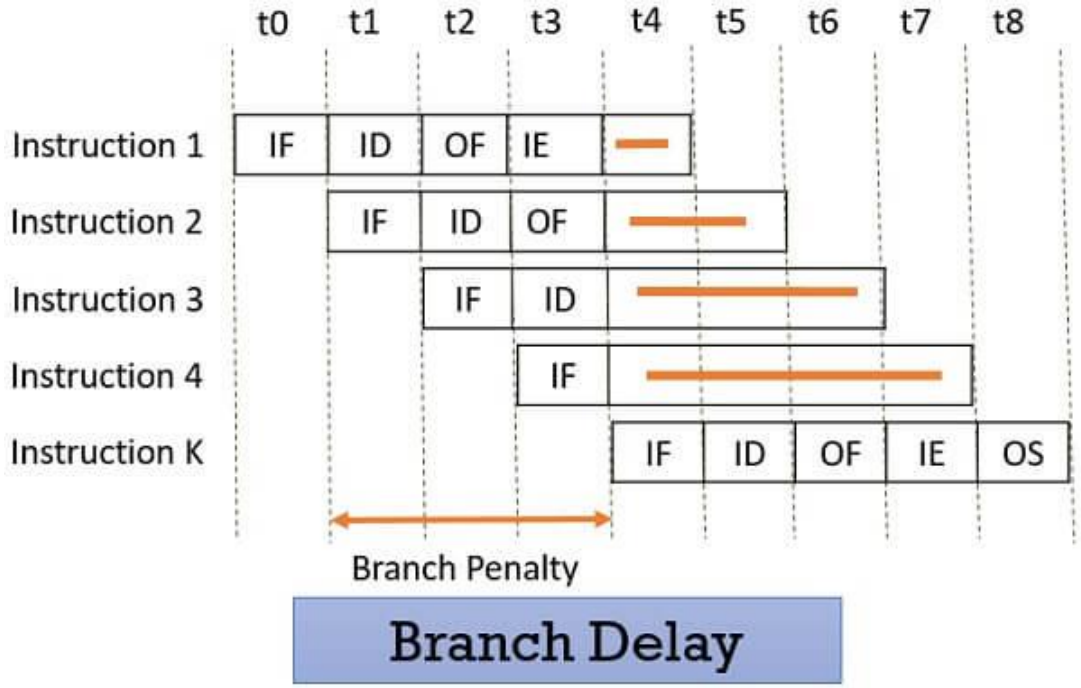
Structural Hazard - Occurrence



Pipelining Hazards

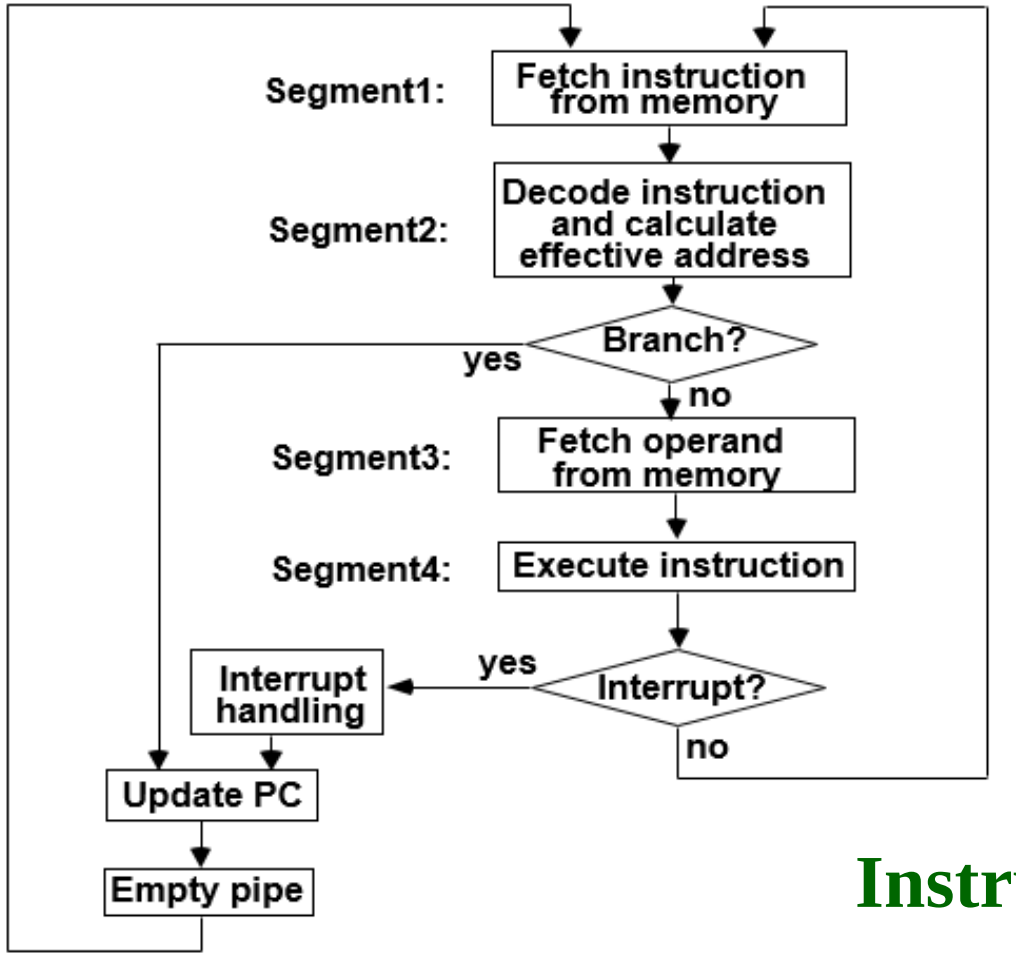
- **Instructional/Control hazards (Branching, Memory delays)** - Branches and other instructions that change the PC make the fetch of the next instruction to be delayed.
  - Eg: Branch target address is not known until the branch instruction is completed





Pipelining Hazards

Instructional/Control hazards (Branching, Memory delays)



Step:		1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction	1	FI	DA	FO	EX									
	2		FI	DA	FO	EX								
(Branch)	3			FI	DA	FO	EX							
	4				FI	-	-	FI	DA	FO	EX			
	5					-	-	-	FI	DA	FO	EX		
	6									FI	DA	FO	EX	
	7										FI	DA	FO	EX

Instruction Execution in a 4-stage Pipeline

Ways to Resolve Data Hazard

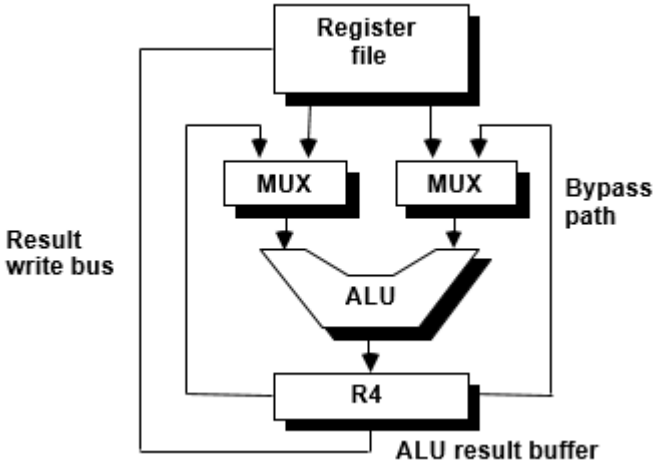
- Data Hazards - resolved by
  - Hardware techniques
    - interlock
    - Operand Forwarding (bypassing, short-circuiting)
  - Software techniques
    - Using NOP instructions
    - Instruction Scheduling(compiler) for delayed load

Hardware techniques:

- Interlock

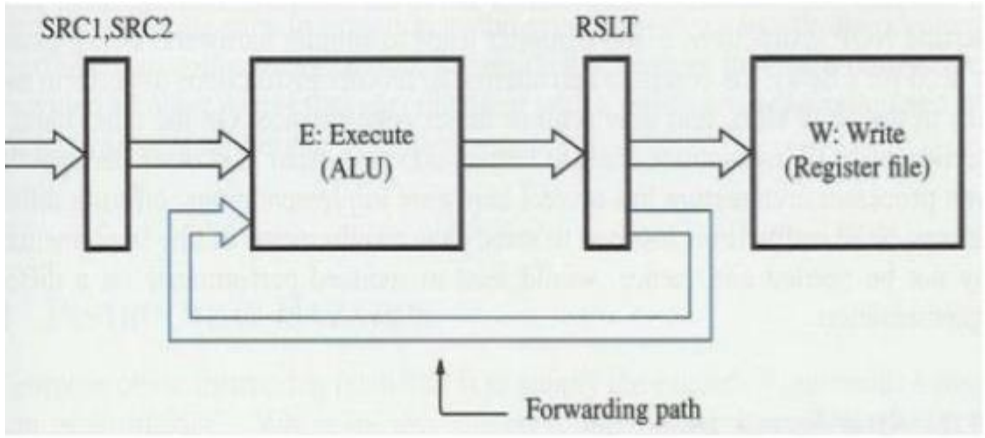
- hardware detects the data dependencies and delays the scheduling of the dependent instruction by stalling enough clock cycles

Ways to Resolve Data Hazard



Hardware techniques:

- Operand Forwarding (bypassing, short-circuiting)
  - Accomplished by a data path that routes a value from a source (usually an ALU) to a user, bypassing a designated register.
  - This allows the value to be produced to be used at an earlier stage in the pipeline than would otherwise be possible



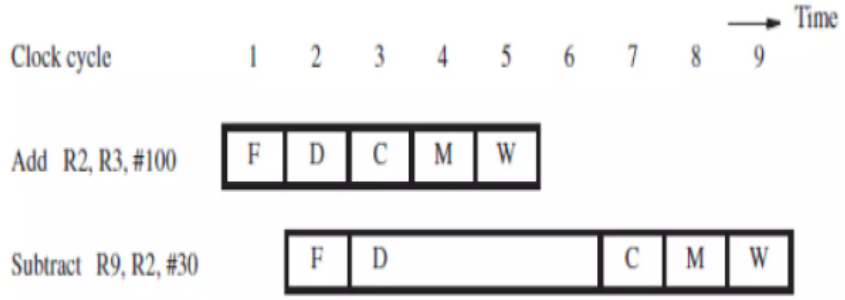


Figure 6.3 Pipeline stall due to data dependency.

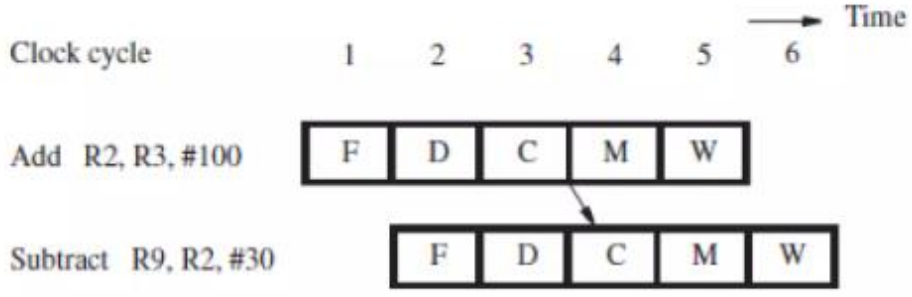


Figure 6.4 Avoiding a stall by using operand forwarding.

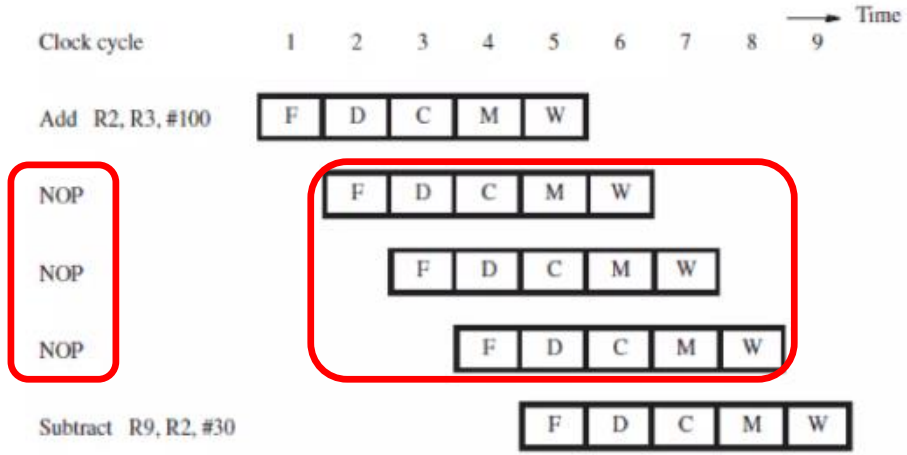
Ways to Resolve Data Hazard

Software techniques:

Using NOP instructions

```
Add       R2, R3, #100
NOP
NOP
NOP
Subtract  R9, R2, #30
```

- Code size increases
- Execution time not reduced



(b) Pipelined execution of instructions

Figure 6.6 Using NOP instructions to handle a data dependency in software.

Instruction Scheduling(compiler) for delayed load

```
a = b + c;
d = e - f;
```

Unscheduled code:

```
    LW    Rb, b
    LW    Rc, c
→   ADD   Ra, Rb, Rc
→   SW    a, Ra
    LW    Re, e
    LW    Rf, f
→   SUB   Rd, Re, Rf
→   SW    d, Rd
```

Scheduled Code:

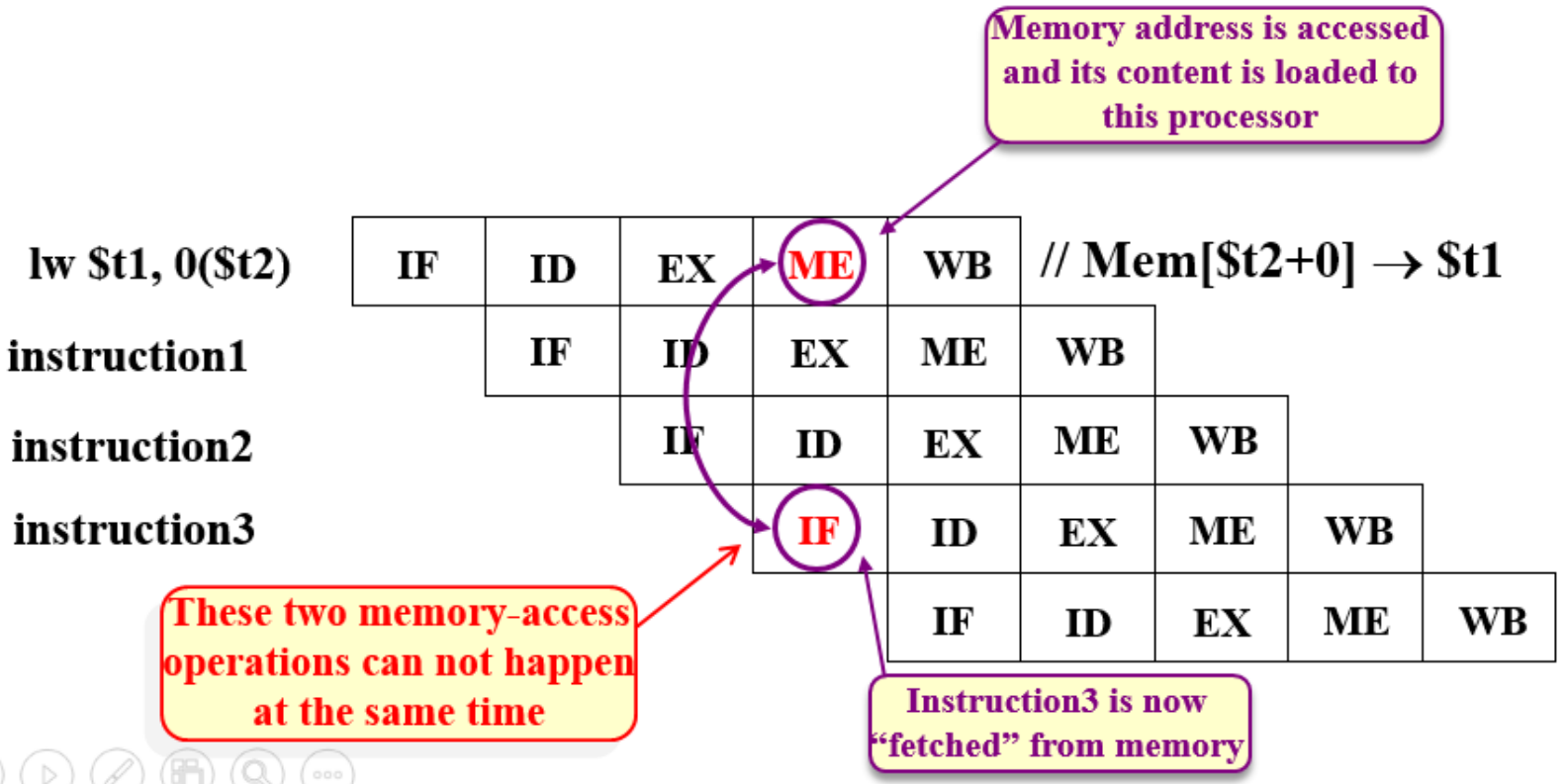
```
    LW    Rb, b
    LW    Rc, c
    LW    Re, e
    ADD   Ra, Rb, Rc
    LW    Rf, f
    SW    a, Ra
    SUB   Rd, Re, Rf
→   SW    d, Rd
```

Delayed Load

A load requiring that the following instruction not use its result

Ways to Resolve Structural Hazard

Structural Hazard - Occurrence

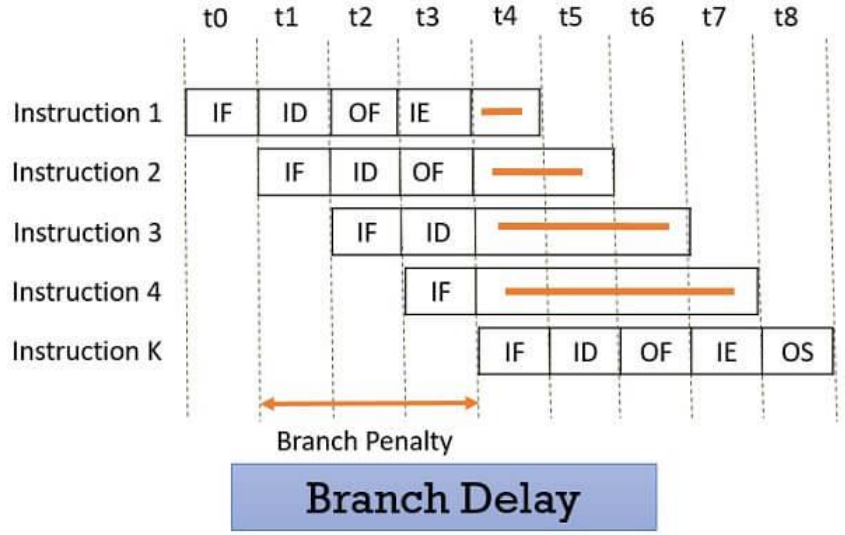


Ways to Handle SH

- Duplicate Resources
- Pipeline the resources
- Reordering the instructions

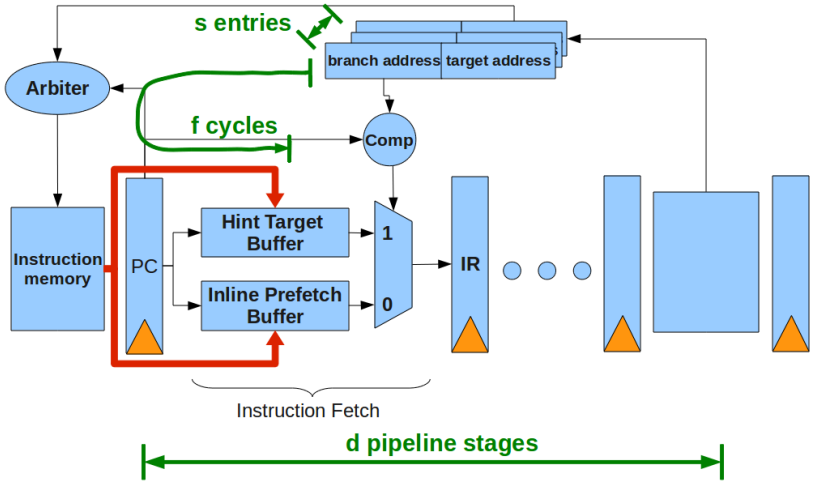
Ways to Resolve Control Hazard

- Branch target address is not known until the branch instruction is completed
- **Handling Control Hazards**
  - Prefetch Target Instruction
  - Branch Target Buffer
  - Loop Buffer
  - Branch Prediction
  - Delayed Branch



Ways to Resolve Control Hazard

- Prefetch Target Instruction
  - Fetch instructions in both streams, branch not taken and branch taken.
  - Both are saved until branch is executed.
  - Then, select the right instruction stream and discard the wrong stream.
- Branch Target Buffer (BTB; Associative Memory)
  - Entry: Address of previously executed branches; Target instruction and the next few instructions.
  - When fetching an instruction, search BTB.
  - If found - fetch the instruction stream in BTB;
  - If not - new stream is fetched and update BTB.
  - The BTB typically has a 90% prediction accuracy and buffer hit rate.



Ways to Resolve Control Hazard

- Loop Buffer (High Speed Register file)
  - Storage of entire loop that allows to execute a loop without accessing memory
- Branch Prediction
  - Guessing the branch condition, and fetch an instruction stream based on the guess.
  - Correct guess eliminates the branch penalty.

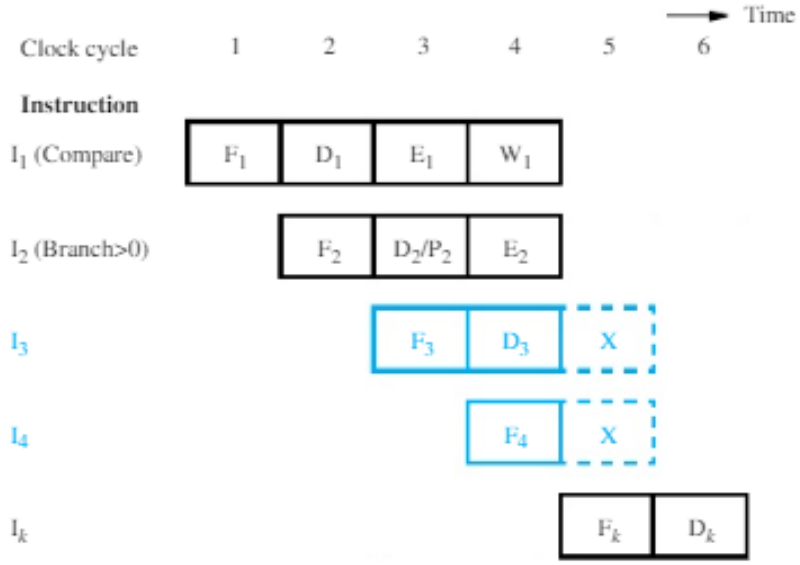


Figure 8.14 Timing when a branch decision has been incorrectly predicted as not taken.

Ways to Resolve Control Hazard

- Delayed Branch
  - Compiler detects the branch and rearranges the instruction sequence by inserting useful instructions that keep the pipeline busy in the presence of a branch instruction

LOOP	Shift_left	R1
	Decrement	R2
	Branch=0	LOOP
NEXT	Add	R1,R3

(a) Original program loop

LOOP	Decrement	R2
	Branch=0	LOOP
	Shift_left	R1
NEXT	Add	R1,R3

(b) Reordered instructions

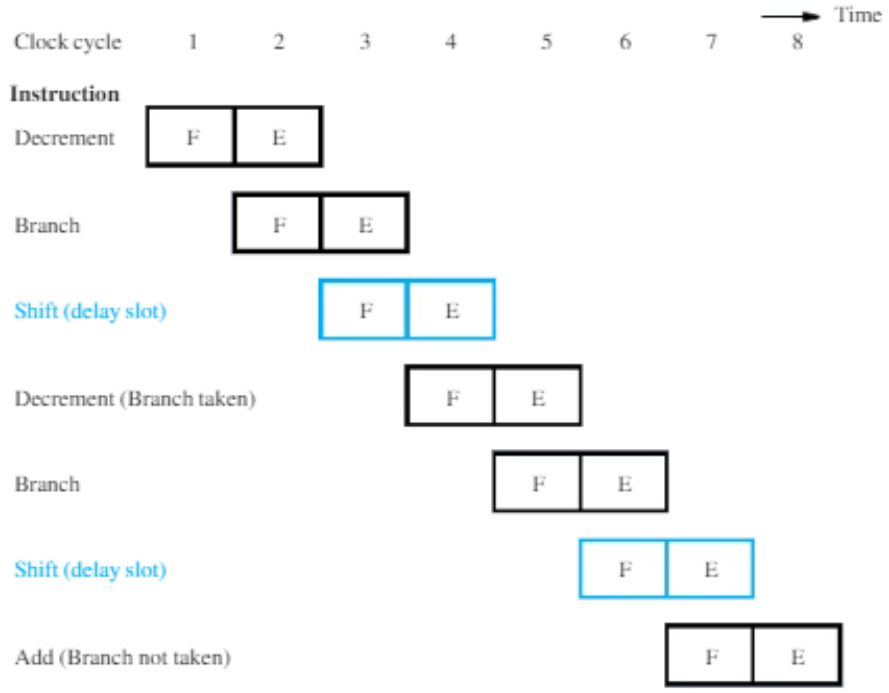


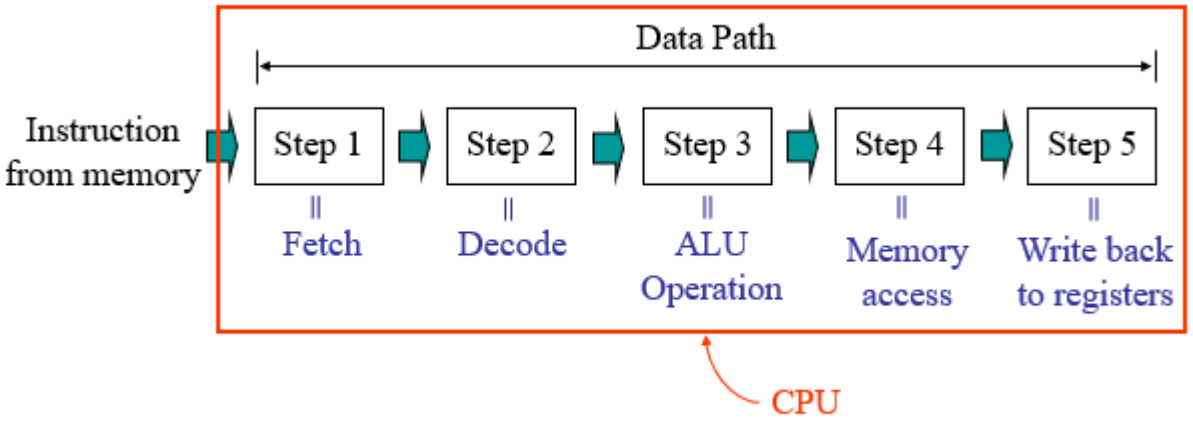
Figure 8.13 Execution timing showing the delay slot being filled during the last two passes through the loop in Figure 8.12b.

BCSE205L
Computer Architecture and Organization

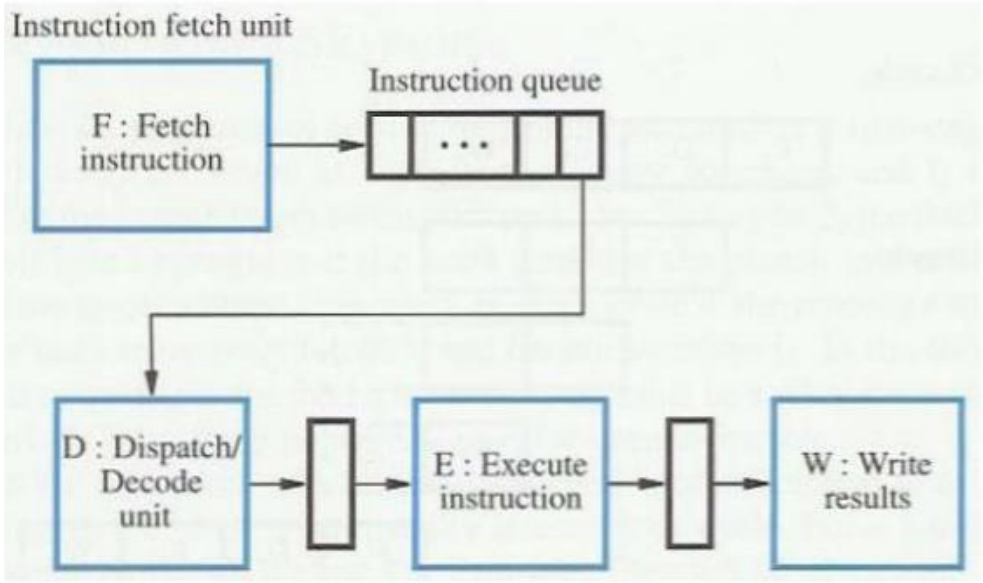
Module 7 – Parallelism – Superscalar Architecture

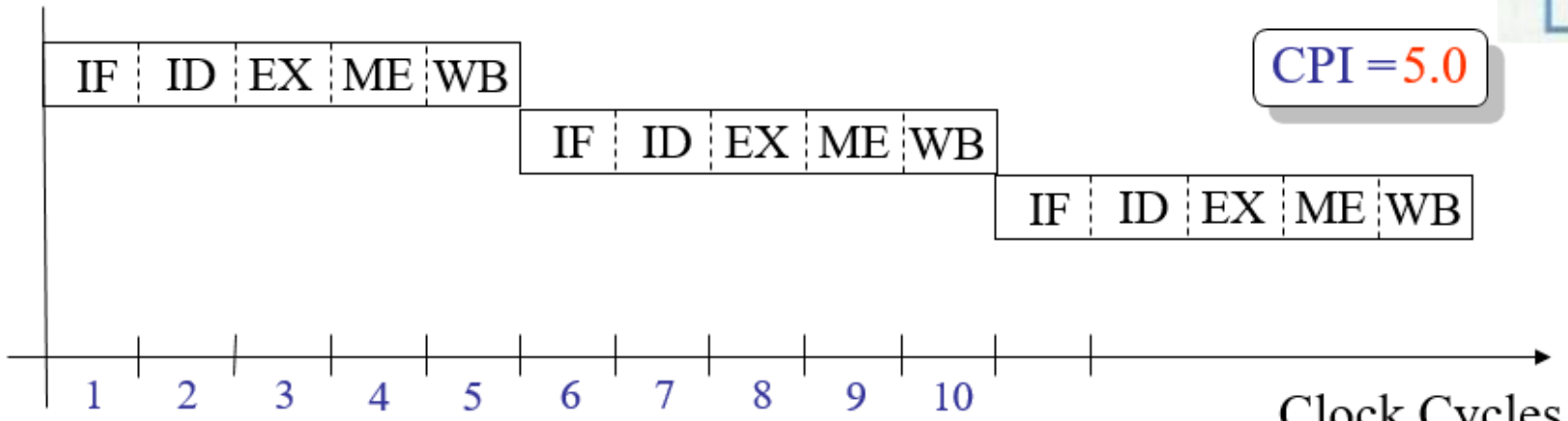
Module:7	High Performance Processors	7 Hours
Classification of models - Flynn's taxonomy of parallel machine models (SISD, SIMD, MISD, MIMD) - Pipelining: Two stages, Multi stage pipelining, Basic performance issues in pipelining, Hazards, Methods to prevent and resolve hazards and their drawbacks - Approaches to deal branches - Superscalar architecture: Limitations of scalar pipelines, superscalar versus super pipeline architecture, superscalar techniques, performance evaluation of superscalar architecture - performance evaluation of parallel processors: Amdahl's law, speed-up and efficiency.		

Scalar Architecture /Non-pipelined Architecture



Datapath: a sequence of CPU circuits that execute CPU instructions step by step



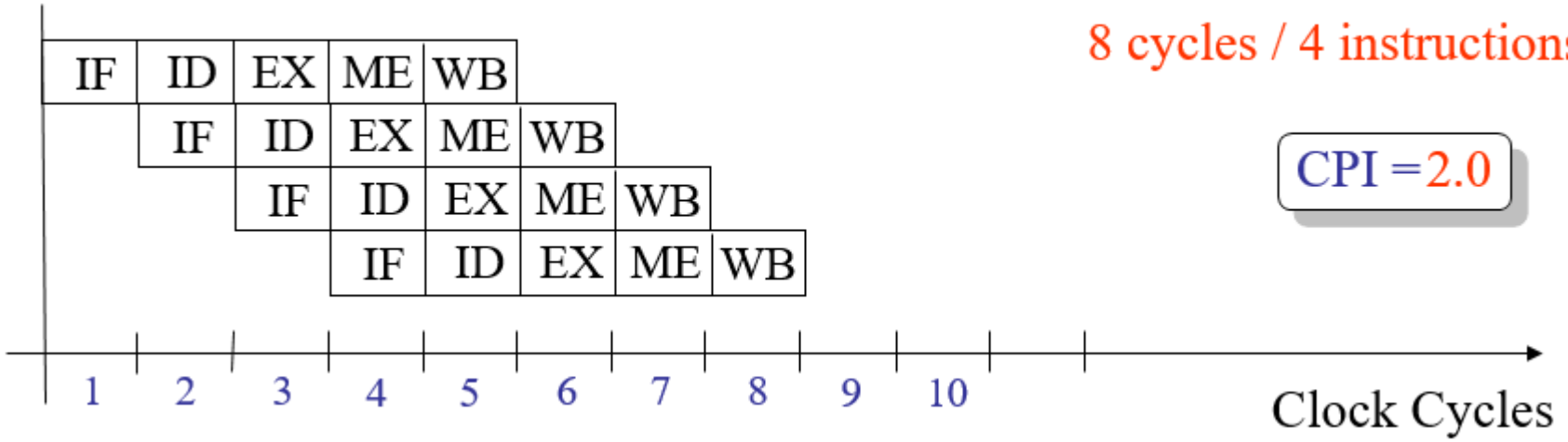


CPI = 5.0

Scalar Pipeline Architecture

In pipelined architecture,

- The hardware of the CPU is split up into several functional units.
- Each functional unit performs a dedicated task.
- The number of functional units may vary from processor to processor.
- These functional units are called as stages of the pipeline.
- Control unit manages all the stages using control signals.
- There is a register associated with each stage that holds the data.
- There is a global clock that synchronizes the working of all the stages.
- At the beginning of each clock cycle, each stage takes the input from its register.
- Each stage then processes the data and feed its output to the register of the next stage.



8 cycles / 4 instructions

CPI =2.0

Limitations of Scalar Pipeline

- Instructions, regardless of type, traverse the same set of pipeline stages.
- Only one instruction can be resident in each pipeline stage at any time.
- Instructions advance through the pipeline stages in a lockstep fashion.
- Upper bound on pipeline throughput.

- Scalar upper bound on throughput
  - IPC <= 1 or CPI >= 1
  - Solution: wide (superscalar) pipeline
- Inefficient unified pipeline
  - Long latency for each instruction
  - Solution: diversified, specialized pipelines
- Rigid pipeline stall policy
  - One stalled instruction stalls all newer instructions
  - Solution: Out-of-order execution, distributed execution pipelines

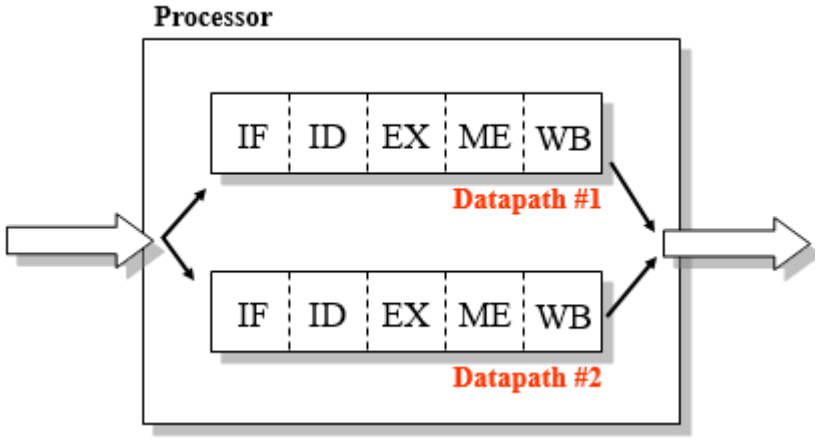
Limitations of Scalar Pipeline

- Processor performance can be increased
  - By increasing instructions per cycle (IPC)
  - By increasing frequency
  - By decreasing the total instruction count

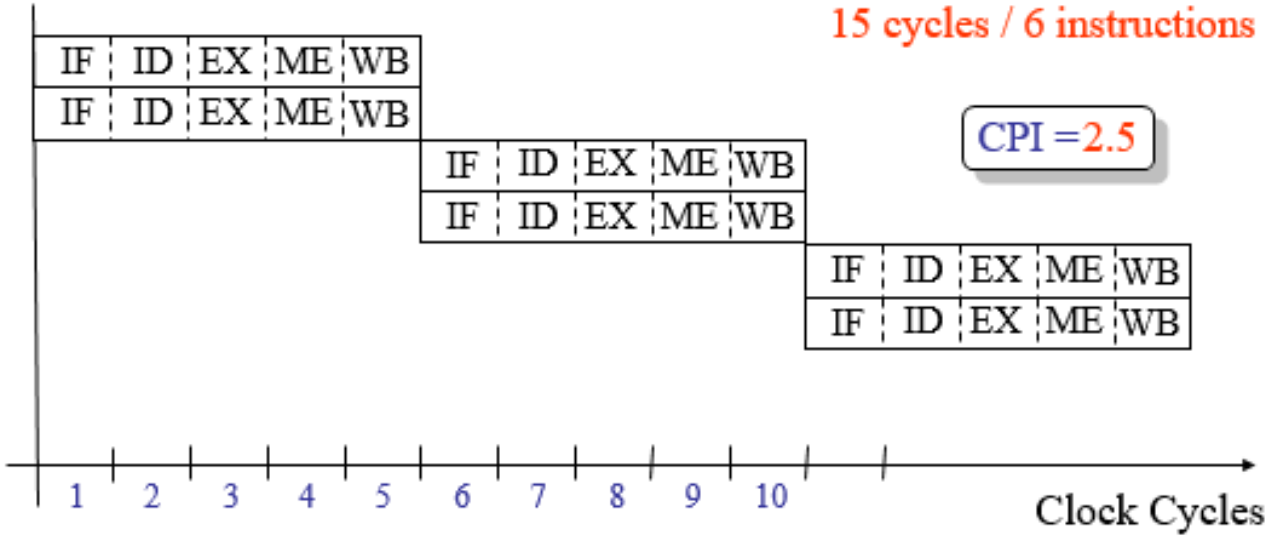
$$Performance = \frac{1}{instruction\ count} \times \frac{instruction}{cycle} \times \frac{1}{cycle\ time}$$

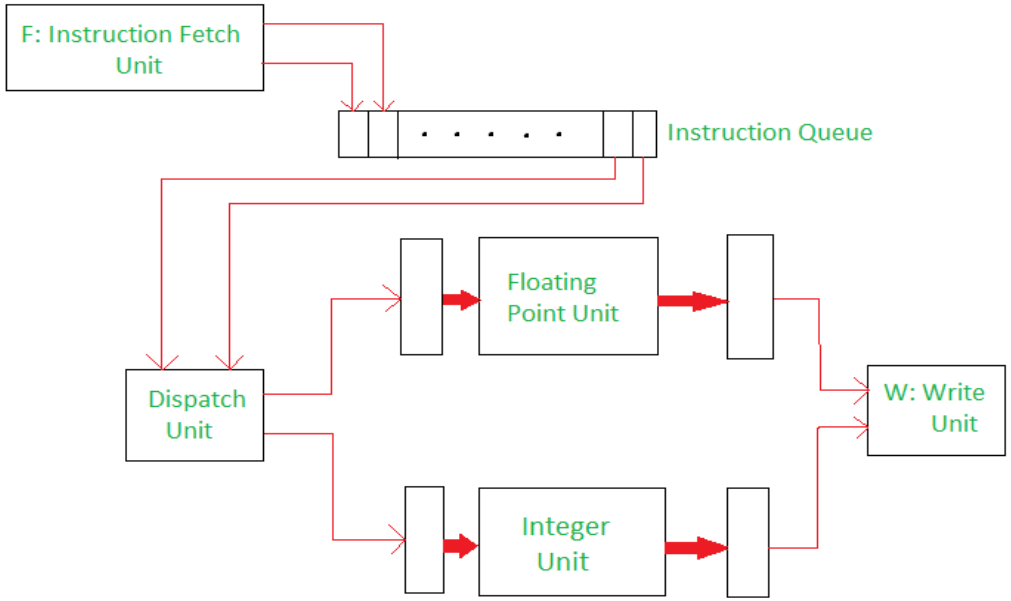
$$= \frac{IPC\ X\ frequency}{instruction\ count}$$

Superscalar Architecture

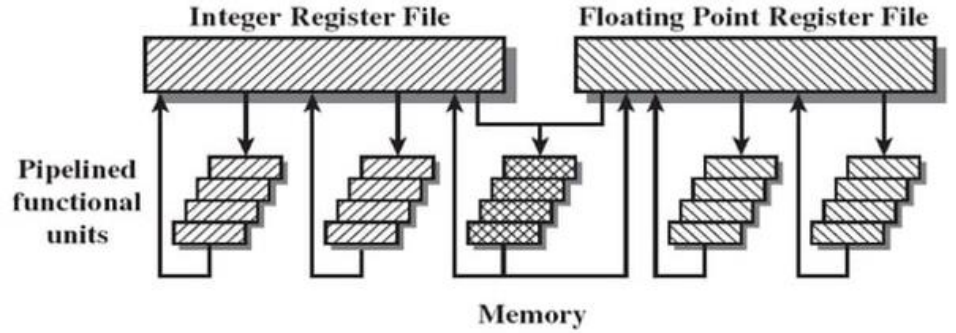


Multi scalar Datapath





Processor with Two Execution Units



Superscalar Architecture

- **Scalar** processors: **one instruction** per cycle
- **Superscalar** : **multiple instruction** pipelines are used.
- Purpose: To exploit more instruction level parallelism in user programs.
- Only independent instructions can be executed in parallel.

- Superscalar machines go beyond just a single-instruction pipeline
  - Able to simultaneously advance multiple instructions through the pipeline stages
  - They incorporate multiple functional units
- Superscalar machines posses the ability to execute instructions in an order different from that specified by the original program
  - The sequential ordering of instructions in standard programs implies some unnecessary precedences between the instructions
- The capability of executing instructions out of program order relieves the sequential imposition
  - It allows more parallel processing of instructions without requiring modification of the original program

Superscalar Techniques

A more aggressive approach is to equip the processor with multiple processing units to handle several instructions in parallel in each processing stage. With this arrangement, several instructions start execution in the same clock cycle and the process is said to use multiple issue. Such processors are capable of achieving an instruction execution throughput of more than one instruction per cycle. They are known as 'Superscalar Processors'.

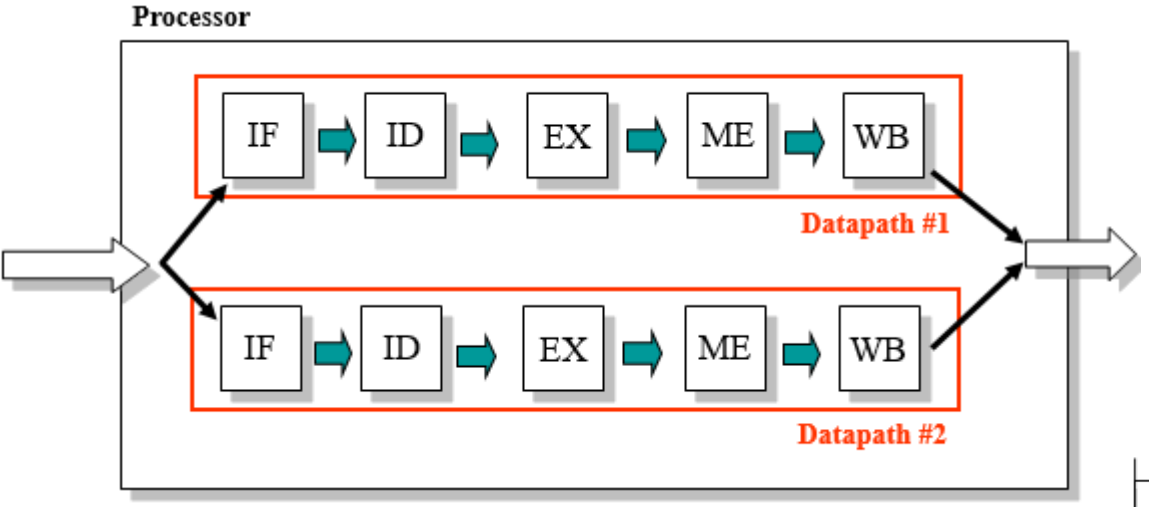
- 1st invented in 1987
- Superscalar processor executes multiple independent instructions in parallel.
- Common instructions (arithmetic, load/store etc) can be initiated simultaneously and executed independently.
- Applicable to both RISC & CISC, but usually in RISC

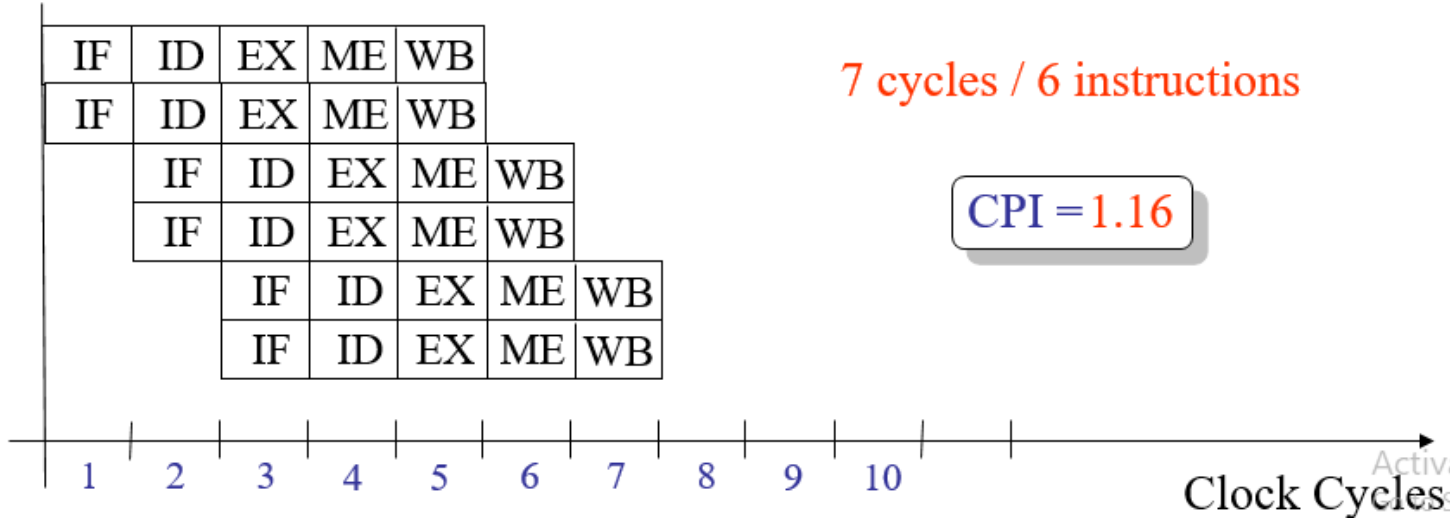
- In superscalar multiple independent instruction pipelines are used. Each pipeline consists of multiple stages, so that each pipeline can handle multiple instructions at a time.
- A superscalar processor typically fetches multiple instructions at a time and then attempts to find nearby instructions that are independent of one another and can therefore be executed in parallel.
- If the input to one instruction depends on the output of a preceding instruction, then the latter instruction cannot complete execution at the same time or before the former instruction.

Superscalar was designed to improve the performance of these operations by executing them concurrently in multiple pipelines

Super Pipeline Architecture

- Combination of Superscalar and Pipeline architectures





7 cycles / 6 instructions

CPI = 1.16

Superscalar Vs Superpipelined Architecture

Superscalar Processors	Superpipelined Processors
• Rely on **spatial** parallelism	• Rely on **temporal** parallelism
• Multiple operations running on **separate hardware** concurrently	• Overlapping multiple operations on a **common hardware**
• Achieved by **duplicating hardware resources** such as execution units and register file ports	• Achieved through more **deeply pipelined execution units** with faster clock cycles
• Requires **more** transistors	• Requires **faster** transistors

- **Temporal parallelism-** known as pipelining - way to execute a task as a series of sub-tasks, with one functional unit performing each sub-task.
- All the successive units can work simultaneously, in an overlapped fashion.

- **Spatial parallelism -** involves multiple tasks that are executed simultaneously, with each unit of information processed by its own dedicated component.

BCSE205L
Computer Architecture and Organization

Module 7 – Performance Evaluation – Amdahl's Law, SpeedUp, Efficiency

Module:7	High Performance Processors	7 Hours
Classification of models - Flynn's taxonomy of parallel machine models (SISD, SIMD, MISD, MIMD) - Pipelining: Two stages, Multi stage pipelining, Basic performance issues in pipelining, Hazards, Methods to prevent and resolve hazards and their drawbacks - Approaches to deal branches - Superscalar architecture: Limitations of scalar pipelines, superscalar versus super pipeline architecture, superscalar techniques, performance evaluation of superscalar architecture - performance evaluation of parallel processors: Amdahl's law, speed-up and efficiency.		

Performance Evaluation of Superscalar & Parallel Processors

- Performance Metrics of Parallel Processors
  - Speedup
  - Efficiency
  - Granularity $Granularity = \frac{Computation\ Time}{Communication\ Time}$
  - load balance

Performance Evaluation – Speed Up

Speedup:

- Speedup metric is a quantitative measure of performance, which defines benefit of running a program in parallel
- ***Speedup is the ratio of the time it takes to execute a program in serial (with one processor) to the time it takes to execute in parallel (with many processors).***
- The formula for speedup:

$$S(n) = \frac{T(1)}{T(n)} = \frac{\text{Time taken to execute the program using single processor}}{\text{Time taken to execute the program using } n \text{ no. of processors}}$$

$$\text{Speedup} = \frac{\textit{Serial Time}}{\textit{Parallel Time}}$$

For example: T(1) = 1sec & if n=2 processors then T(2) = ½= 0.5 sec

Hence **S(n)** = T(1)/T(2) =1/0.5 = 2 means *Speedup increases by 2 times*

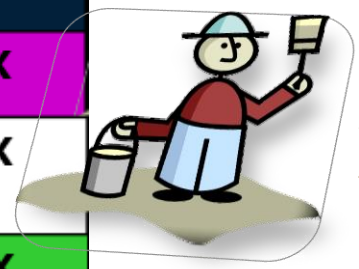
Performance Evaluation – Speed Up

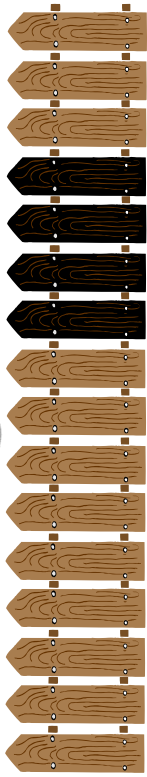
- **Example: Painting a picket fence**
  - 30 minutes of preparation (serial)
  - One minute to paint a single picket
  - 30 minutes of cleanup (serial)
- Thus, 300 pickets takes 360 minutes if processed in serial.

$$\text{Speedup} = \frac{\text{Serial Time}}{\text{Parallel Time}}$$

- For N=1 (Serial) => SpeedUp = 360/360=1
- For N=2 (Parallel) => SpeedUp = 360/210 = 1.7
- For N=10 (Parallel) => SpeedUp = 360/90 = 4
- ……

Number of painters	• 30 minutes (serial) • 1 minute to paint a single picket (300 pickets) • 30 minutes (serial) Time	Speedup
1	30 + 300 + 30 = 360	1.0X
2	30 + 150 + 30 = 210	1.7X
10	30 + 30 + 30 = 90	4.0X
100	30 + 3 + 30 = 63	5.7X
Infinite	30 + 0 + 30 = 60	6.0X





Performance Evaluation – Efficiency

- Efficiency
  - Measure of how effectively computation resources (threads) are kept busy

$$\text{Efficiency} = \frac{Speedup}{Numbe\ of\ Threads} \text{ x } 100$$

- For N=1 (Serial) => Efficiency = 1/1*100 =100
- For N=2 (Parallel) => Efficiency = 1.7/2 *100 =85
- For N=10 (Parallel) => Efficiency = 4/10 *100 =40
- …….

Number of painters	• 30 minutes (serial) • 1 minute to paint a single picket (300 pickets) • 30 minutes (serial) Time	Speedup	Efficiency
1	30 + 300 + 30 = 360	1.0X	100%
2	30 + 150 + 30 = 210	1.7X	85%
10	30 + 30 + 30 = 90	4.0X	40%
100	30 + 3 + 30 = 63	5.7X	5.7%
Infinite	30 + 0 + 30 = 60	6.0X	very low

Performance Evaluation of Parallel Processors – Amdahl's Law

- Amdahl's Law - SpeedUp Performance Law

"*Law governing the speedup of using parallel processors on a problem, versus using only one serial processor, under the assumption that the problem size remains the same when parallelized*".

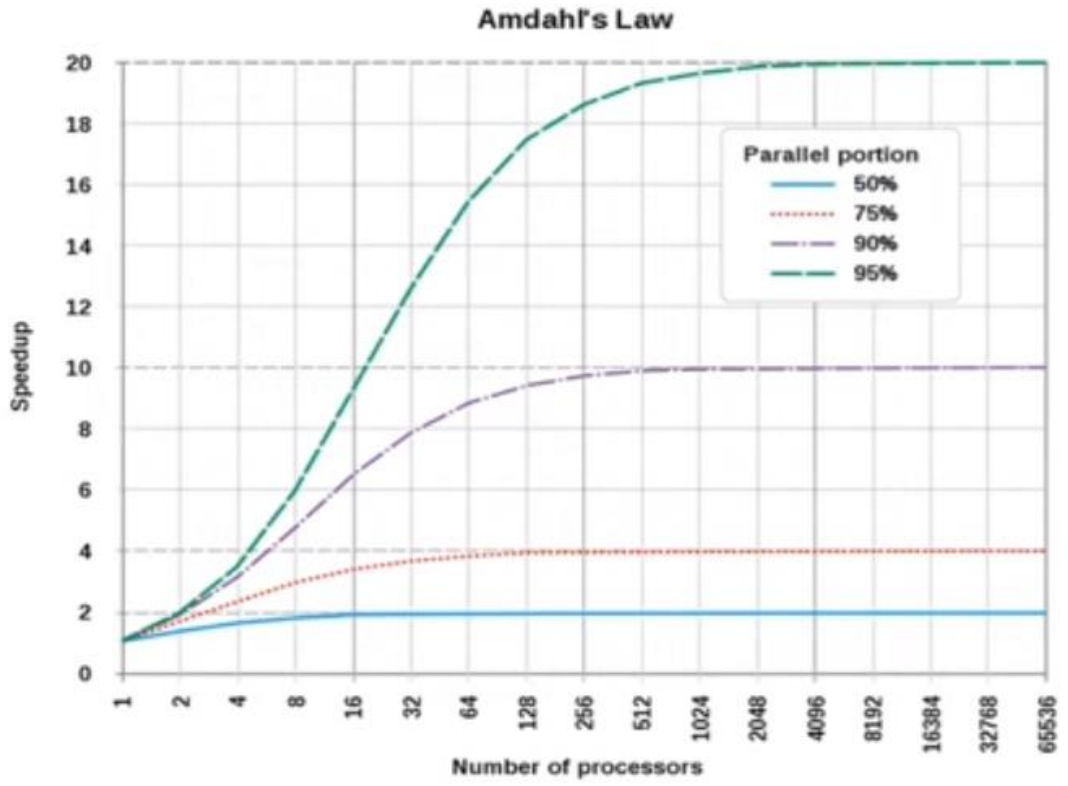
Performance Evaluation of Parallel Processors – Amdahl's Law

- The potential ***speedup*** of an algorithm on parallel computing platform is given by Amdahl's Law; originally formulated by *Gene Amdahl's* on 1960's
- It is one of the **speedup performance law**
- It is based on a **fixed problem size (or fixed work load)**
- Actually *to keep the efficiency of system fixed* we have to increase both the size of the problem and the no. of processors simultaneously
- **Amdahl's Law** tells that for a given problem size, *the* ***speedup doesn't increase linearly*** **as the number of processor increases. In fact, speed-up tends to become *saturated***
- **Amdahl's law** states that a small portion of the program which cannot be parallelized (serial part), will limit the overall speed-up, available from parallelization
  - Typically, any large mathematical or engineering problem consists of several parallelizable parts & several serial (non-parallelizable) parts

Computation Problem = Serial Part + Parallel Part

Performance Evaluation of Parallel Processors – Amdahl's Law

- *The **speedup** of a program using multiple processors in parallel computing is **limited** by the time needed for the **sequential fraction (non-parallelizable part) of the program**.*



Performance Evaluation of Parallel Processors – Amdahl's Law

- SpeedUp – Ratio of the time it takes to execute a program in serial(with one processor) to the time it takes to execute in parallel (with many processors)

$$\text{Speedup} = \frac{\text{Time to execute program on a single processor}}{\text{Time to execute program on } N \text{ parallel processors}}$$

- Let

f = fraction of the execution time that involves code that is infinitely parallelizable with no scheduling overhead.

(1-f) = fraction of the execution time that involves code that is inherently sequential

T = Total execution time of the program using a single processor

Then,

$$\text{Speedup} = \frac{T(1-f) + Tf}{T(1-f) + \frac{Tf}{N}}$$

$$= \frac{1}{(1-f) + \frac{f}{N}}$$

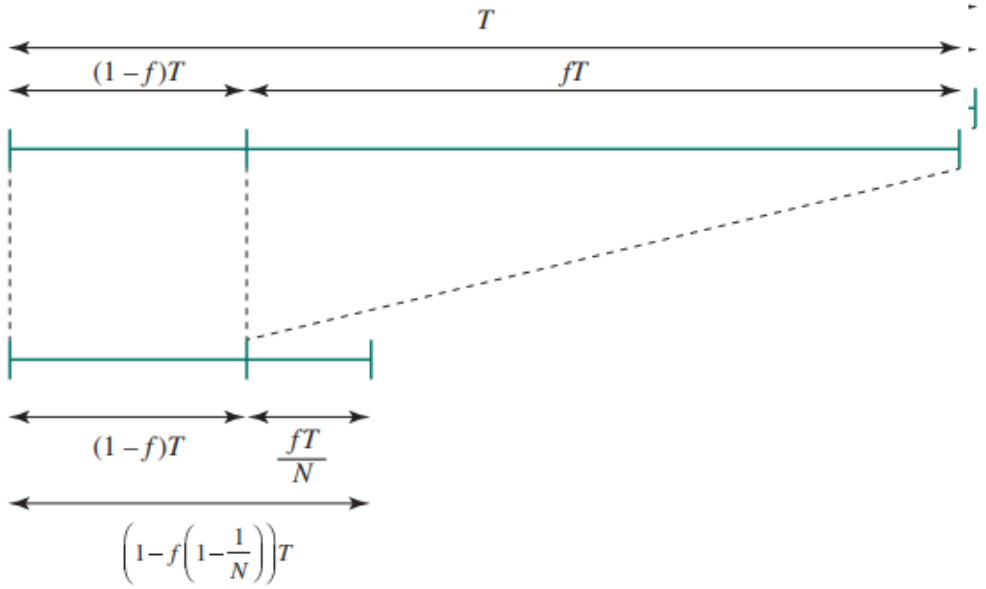


Illustration of Amdahl's Law

Performance Evaluation of Parallel Processors – Amdahl's Law

- Two Important Conclusions drawn
  - When *f is small*, the use of parallel processors has little effect.
  - As *N approaches infinity*, speedup is bound by 1/(1 - f), so that there are diminishing returns for using more processors.
- Amdahl's law can be generalized to evaluate any design or technical improvement in a computer system.
- Consider any enhancement to a feature of a system that results in a speedup.

$$\text{Speedup} = \frac{T(1-f)+Tf}{T(1-f)+\frac{Tf}{N}} = \frac{1}{(1-f)+\frac{f}{N}}$$

The speedup can be expressed as

$$\text{Speedup} = \frac{\text{Performance after enhancement}}{\text{Performance before enhancement}} = \frac{\text{Execution time before enhancement}}{\text{Execution time after enhancement}}$$

Performance Evaluation of Parallel Processors – Amdahl's Law

- The speedup can be expressed as

$$\text{Speedup} = \frac{\text{Performance after enhancement}}{\text{Performance before enhancement}} = \frac{\text{Execution time before enhancement}}{\text{Execution time after enhancement}}$$

$$\text{Speedup} = \frac{T(1-f) + Tf}{T(1-f) + \frac{Tf}{N}} = \frac{1}{(1-f) + \frac{f}{N}}$$

- Suppose that a feature of the system is enhanced to improve the performance.
  - Let ***f =*** a fraction of the time before enhancement ***(fE)***
  - ***Su_f =*** speedup of that feature after enhancement
  - Then the overall speedup

of the system is

$$\text{overall Speedup} = \frac{1}{(1-f) + \frac{f}{SU_f}}$$

Amdahl's Law – for Fraction Enhancement

Consider ***fE*** – Fraction Enhanced ***(f)***
1-fE – Unaffected Fraction ***(1-f)***
fI = Factor of Improvement ***(SU_f or N)***
Then the overall speedup of the system is

$$S = ((1 - fE) + (fE / fI))^{-1}$$

Performance Evaluation – Problems

Problem 1

70% of a program's execution time occurs inside a loop that can be executed in **parallel** and rest **30%** in **serial.** What is the maximum speedup we should expect from a parallel version of the program executing on 8 CPUs?

f = 0.7
1-f = 0.3
N=8

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

$$\text{Speedup} = \frac{1}{0.3+\frac{0.7}{8}} = \frac{1}{0.3+0.0875} = \frac{1}{0.3875} = 2.6$$

Performance Evaluation – Problems

Problem 2

80% of a program's execution time occurs inside a loop that can be executed in **parallel** and rest **20%** in **serial.** What is the maximum speedup we should expect from a parallel version of the program executing on 8 CPUs?

f = 0.8
1-f = 0.2
N=8

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

$$\text{Speedup} = \frac{1}{0.2+\frac{0.8}{8}} = \frac{1}{0.2+0.1} = \frac{1}{0.3} = 3.33$$

Performance Evaluation – Problems

Problem 3

95% of a program's execution time occurs inside a loop that can be executed in **parallel** and rest **5%** in **serial.** What is the maximum speedup we should expect from a parallel version of the program executing on 8 CPUs?

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

f = 0.95
1-f = 0.05
N=8

$$\text{Speedup} = \frac{1}{0.05 + \frac{0.95}{8}} = 5.9$$

Performance Evaluation – Problems

Problem 4

We have 4 processors and only 10% of the code is parallelizable. Find the speed up.

f = 10% = 0.1
1-f = 0.9
N=4

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

$$\text{Speedup} = \frac{1}{0.9+\frac{0.1}{4}} = \frac{1}{0.9+0.025} = \frac{1}{0.925} = 1.081$$

Performance Evaluation – Problems – Overall Speedup

Problem 5 – Finding Overall speed up - Given, speedup of the enhanced machine.

Floating point instructions are improved to run twice as fast, but only 10% of the time was spent on these instructions originally. How much faster is the new machine?

f = 10% = 0.1

1-f = 0.9

Speedup of enhanced machine (SU_f)=2

$$\text{overall Speedup} = \frac{1}{(1-f) + \frac{f}{SU_f}}$$

$$\text{Speedup} = \frac{1}{0.9+\frac{0.1}{2}} = \frac{1}{0.9+0.05} = \frac{1}{0.95} = 1.053$$

How much faster would the new machine be if floating point instructions become 100 times faster?

$$\text{Speedup} = \frac{1}{0.9+\frac{0.1}{100}} = 1.109$$

Problem 6

Problem Type 1 – **Predict System Speedup**

Example: Let a program have 40 percent of its code enhanced (so fE = 0.4) to run 2.3 times faster (so fI = 2.3). What is the overall system speedup S?

Solution:

- Step 1: Setup the equation: S = ((1 – fE) + (fE / fI))^-1
- Step 2: Plug in values & solve
  - S = ((1 – 0.4) + (0.4 / 2.3))-1 = (0.6 + 0.174)^-1
  - =1 / 0.774 = 1.292

Hint: (Similarity)

Consider

- ***fE*** – Fraction Enhanced ***(f)***
- ***1-fE*** – Unaffected Fraction ***(1-f)***
- ***fI*** = Factor of Improvement ***(SU_f or N)***

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

Amdahl's Law – for Fraction Enhancement

Consider ***fE*** – Fraction Enhanced ***(f)***
1-fE – Unaffected Fraction ***(1-f)***
fI = Factor of Improvement ***(SU_f or N)***
Then the overall speedup of the system is

S = ((1 – fE) + (fE / fI))^-1

Problem 7

Problem Type 2 – **Predict Speedup of Fraction Enhanced**

Example: Let a program have 40 percent of its code enhanced (so fE = 0.4) to yield a system speedup 4.3 times faster (so S = 4.3). What is the factor of improvement fI of the portion enhanced?

Solution:

Case #1: Can we do this? In other words, let's determine if by enhancing 40 percent of the system, it is possible to make the system go 4.3 times faster

Step 1: Assume the limit, where fI = infinity, so S = ((1 – fE) + (fE / fI))^-1
S = 1 / (1 – fE)

Step 2: Plug in values & solve S = ((1 – 0.4))-1 = 1 / 0.6 = 1.67 .

Step 3: So S = 1.67 is the maximum possible speedup, and we cannot achieve S = 4.3 !!

Consider ***fE*** – Fraction Enhanced ***(f)***

- ***1-fE*** – Unaffected Fraction ***(1-f)***
- ***fI*** = Factor of Improvement ***(SU_f or N)***

$$\text{Speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$

Problem 8

A different case: Let's determine if by enhancing 40 percent of the system, it is possible to make the system go 1.3 times faster …

Step 1: Assume the limit, where f_I = infinity, so $S = ((1 - f_E) + (f_E / f_I))^{-1} \rightarrow S = 1 / (1 - f_E)$

Step 2: Plug in values & solve $S = ((1 - 0.4))^{-1} = 1 / 0.6 = \mathbf{1.67}$.

Step 3: So S = 1.67 is the **maximum possible speedup**, and we <u>can</u> achieve S = 1.3 !!

Step 4: Solve speedup equation for f_I :

$1/S = (1 - f_E) + (f_E / f_I)$ [invert both sides]

$1/S - (1 - f_E) = f_E / f_I$ [subtract $(1 - f_E)$]

$(1/S - (1 - f_E))^{-1} = f_I / f_E$ [invert both sides]

$f_E \cdot (1/S - (1 - f_E))^{-1} = f_I$ [multiply by f_E]

Step 5: Plug in values & solve:

$f_I = f_E \cdot (1/S - (1 - f_E))^{-1}$

$= 0.4 \cdot (1/1.3 - (1 - 0.4))^{-1}$

$= 0.4 / (0.769 - 0.6) = \mathbf{2.367}$

Step 6: Check your work: $S = ((1 - f_E) + (f_E / f_I))^{-1} = (0.6 + (0.4/2.367))^{-1} = 1.3$ ✓

Hint: (Similarity)

Consider ***fE*** – Fraction Enhanced ***(f)***

- ***1-fE*** – Unaffected Fraction ***(1-f)***
- ***fI*** = Fraction of Enhancement ***(SU_f or N)***

$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{N}}$$

Problem 9

Problem Type 3 – **Predict Fraction of System to be Enhanced**

Example: Let a program have a portion fE of its code enhanced to run 4 times faster (so fI = 4), to yield a system speedup 3.3 times faster (so S = 3.3). What is the fraction enhanced (fE)?

Step 1: *Can this be done*? Assuming f_I = infinity, $S = 3.3 = ((1 - f_E))^{-1}$ so minimum $f_E = 0.697$

Yes, this can be done for maximum f_I, so let's solve the equation to determine actual f_E

Step 2: Solve speedup equation for f_E :

$S = ((1 - f_E) + (f_E / f_I))^{-1}$	[state the equation]
$3.3 = ((1 - f_E) + (f_E / 4))^{-1}$	[plug in values]
$(1 - f_E) + f_E/4 = 1/3.3 = 0.303$	[invert both sides]
$1 - 0.75 f_E = 0.303$	[regroup]
$0.75 f_E = 1 - 0.303 = 0.697$	[commutativity]
$f_E = 0.697 / 0.75 = \mathbf{0.929}$	[divide by 0.75]

Step 3: Check your work: $S = ((1 - f_E) + (f_E / f_I))^{-1} = (0.071 + (0.929/4))^{-1} = 3.3$ ✓

Hint: (Similarity)

$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{N}}$$

Consider ***fE*** – Fraction Enhanced ***(f)***

- ***1-fE*** – Unaffected Fraction ***(1-f)***
- ***fI*** = Fraction of Enhancement ***(SU_f or N)***

Problem 10

Performance Enhancement Example

- **For the RISC machine with the following instruction mix given earlier:**

Op	Freq	Cycles	CPI(i)	% Time
ALU	50%	1	.5	23%
Load	20%	5	1.0	45%
Store	10%	3	.3	14%
Branch	20%	2	.4	18%

CPI = 2.2

From a previous example

- **If a CPU design enhancement improves the CPI of load instructions from 5 to 2, what is the resulting performance improvement from this enhancement:**

Fraction enhanced = F = 45% or .45

Unaffected fraction = 1- F = 100% - 45% = 55% or .55

Factor of enhancement = S = 5/2 = 2.5

Using Amdahl's Law:

$$\text{Speedup(E)} = \frac{1}{(1 - F) + F/S} = \frac{1}{.55 + .45/2.5} = 1.37$$

Problem 11

- **A program runs in 100 seconds on a machine with multiply operations responsible for 80 seconds of this time. By how much must the speed of multiplication be improved to make the program four times faster?**

$$\text{Desired speedup} = 4 = \frac{100}{\text{Execution Time with enhancement}}$$

→ Execution time with enhancement = 100/4 = 25 seconds

25 seconds = (100 - 80 seconds) + 80 seconds / S

25 seconds = 20 seconds + 80 seconds / S

→ 5 = 80 seconds / S

→ S = 80/5 = 16

Alternatively, it can also be solved by finding enhanced fraction of execution time:

$$F = 80/100 = .8$$

and then solving Amdahl's speedup equation for desired enhancement factor S

$$\text{Speedup(E)} = \frac{1}{(1 - F) + F/S} = 4 = \frac{1}{(1 - .8) + .8/S} = \frac{1}{.2 + .8/s}$$

Solving for S gives S= 16

Hence multiplication should be 16 times faster to get an overall speedup of 4.

Problem 12

- For the previous example with a program running in 100 seconds on a machine with multiply operations responsible for 80 seconds of this time. By how much must the speed of multiplication be improved to make the program five times faster?

$$\text{Desired speedup} = 5 = \frac{100}{\text{Execution Time with enhancement}}$$

→ Execution time with enhancement = 100/5 = 20 seconds

20 seconds = (100 - 80 seconds) + 80 seconds / s
20 seconds = 20 seconds + 80 seconds / s

→ 0 = 80 seconds / s

No amount of multiplication speed improvement can achieve this.